

Universidad de La Habana
Facultad de Matemática y Computación



Título de la tesis

Autor:

Nombre del autor

Tutores:

Nombre del primer tutor

Nombre del segundo tutor

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

Fecha

github.com/noelpc03/Tesis

Dedicación

Agradecimientos

Agradecimientos

Opinión del tutor

Opiniones de los tutores

Resumen

Esta tesis aborda el problema de recuperar automáticamente las expresiones analíticas de todas las soluciones de un sistema de ecuaciones no lineales paramétrico. A partir de una cuadrícula de parámetros generada mediante producto cartesiano, se resuelve numéricamente el sistema para cada tupla utilizando múltiples puntos iniciales, construyendo así un conjunto de datos que contiene todas las soluciones posibles. La principal dificultad reside en que para un mismo valor de los parámetros pueden coexistir varias soluciones distintas, cuyos puntos se entremezclan en el conjunto de entrenamiento. Se propone un algoritmo iterativo de regresión simbólica que descubre secuencialmente cada una de las funciones solución, retirando los puntos ya explicados antes de la siguiente iteración. Se comparan tres funciones de pérdida (error cuadrático medio, conteo de coincidencias y pérdida sigmoideal) sobre 30 ecuaciones univariadas, seleccionando la sigmoideal por su mayor tasa de éxito (90 %) y menor tiempo de cómputo. La metodología completa se evalúa sobre 30 sistemas de dos variables y hasta tres parámetros, logrando recuperar todas las soluciones en el 90 % de los casos, incluyendo sistemas con tres y cuatro expresiones diferentes para una misma tupla de parámetros. Los resultados demuestran que la combinación de la búsqueda iterativa con la pérdida sigmoideal permite obtener expresiones analíticas válidas y de alta precisión, facilitando el análisis de sensibilidad y la predicción del comportamiento del sistema sin necesidad de resolverlo numéricamente cada vez.

Palabras clave: Sistemas de ecuaciones no lineales paramétricos, regresión simbólica, programación genética, descubrimiento de funciones, solución múltiple, pérdida sigmoideal, PySR.

Abstract

This thesis addresses the problem of automatically retrieving analytical expressions for all solutions of a parametric system of nonlinear equations. Starting from a grid of parameters generated through a Cartesian product, the system is solved numerically for each tuple using multiple initial points, thereby constructing a dataset containing all possible solutions. The main challenge lies in the fact that for the same parameter values, multiple distinct solutions may coexist, with their points intermingling in the training set. An iterative symbolic regression algorithm is proposed that sequentially discovers each solution function, removing already explained points before the next iteration. Three loss functions (mean squared error, match count, and sigmoidal loss) are compared across 30 univariate equations, with the sigmoidal function selected for its higher success rate (90 %) and lower computational time. The complete methodology is evaluated on 30 two-variable systems with up to three parameters, successfully recovering all solutions in 90 % of cases, including systems with three and four different expressions for a given parameter tuple. The results demonstrate that combining iterative search with sigmoidal loss enables obtaining valid and highly accurate analytical expressions, facilitating sensitivity analysis and behavior prediction of the system without needing to solve it numerically each time.

Keywords: Parametric nonlinear equation systems, symbolic regression, genetic programming, function discovery, multiple solutions, sigmoidal loss, PySR.

Índice general

Introducción	1
1. Preliminares	4
1.1. Sistemas de ecuaciones no lineales paramétricos	4
1.1.1. Métodos numéricos para el cálculo de ceros	5
1.1.2. Distancia euclidiana	7
1.1.3. Producto cartesiano	8
1.2. Regresión simbólica	9
1.2.1. Concepto y objetivos	9
1.2.2. Fundamentos de programación genética	10
1.2.3. Funciones de pérdida	11
1.2.4. Parámetros del algoritmo de regresión simbólica	12
1.3. Prueba t pareada	13
2. Propuesta de solución	15
2.1. Planteamiento general del problema	15
2.2. Generación de datos de entrenamiento	16
2.2.1. Muestreo del espacio de parámetros	16
2.2.2. Resolución numérica del sistema	17
2.2.3. Filtrado de soluciones duplicadas	17
2.3. Estrategia de regresión simbólica	18
2.3.1. Separación por coordenadas	18
2.3.2. Búsqueda iterativa sobre x_1	20
2.3.3. Extensión a las demás coordenadas	22
2.3.4. Validación de una solución completa	23
2.3.5. Funciones de pérdida utilizadas	24
2.4. Pseudocódigo del proceso principal	25
3. Experimentación y resultados	27
3.1. Diseño experimental	27
3.2. Comparación de funciones de pérdida con ecuaciones individuales . .	29

3.2.1.	Ecuaciones de prueba	29
3.2.2.	Funciones de pérdida evaluadas	30
3.2.3.	Resultados	31
3.2.4.	Análisis	32
3.3.	Extensión a sistemas de ecuaciones	33
3.3.1.	Sistemas de prueba	33
3.3.2.	Configuración específica	35
3.3.3.	Resultados	36
3.3.4.	Análisis	37
3.4.	Consideraciones finales	39
4.	Conclusiones	40
	Recomendaciones	42
A.	Anexos	43
A.1.	Detalles de implementación	43
A.2.	Descripción de los módulos	43
A.3.	Entradas, parámetros y salidas del sistema	44
	Bibliografía	46

Introducción

El estudio de sistemas de ecuaciones no lineales paramétricos es una herramienta fundamental en la modelación de fenómenos científicos y de ingeniería [1]. Desde la determinación de puntos de equilibrio en modelos dinámicos [2] hasta el análisis de circuitos electrónicos [3] o la cinética de reacciones químicas [4], estos sistemas permiten representar cómo interactúan las variables de un proceso en función de ciertos parámetros. En todos estos casos, conocer los valores de las incógnitas que anulan el sistema —sus ceros o soluciones— resulta imprescindible para entender el comportamiento del fenómeno modelado y tomar decisiones a partir de él [5].

Más allá de obtener soluciones numéricas para valores concretos de los parámetros, interesa saber cómo varía cada solución cuando los parámetros cambian dentro de un rango determinado. Esa dependencia funcional permite realizar análisis de sensibilidad [6], predecir el comportamiento del sistema ante nuevas condiciones [7] y evitar el costo computacional de resolver el sistema desde cero para cada nueva tupla de parámetros [8]. En este contexto, el propósito ya no es un valor numérico puntual, sino una expresión analítica que describa cada solución en función de los parámetros.

Desde el punto de vista numérico, la resolución de sistemas no lineales ha sido estudiada en profundidad [9]. Los métodos iterativos clásicos [10] permiten encontrar una solución aislada cuando se dispone de una aproximación inicial adecuada, pero no garantizan la obtención de todas las soluciones posibles [11]. Para abordar esta limitación se han desarrollado estrategias capaces de enumerar el conjunto completo de raíces reales en un dominio acotado, entre ellas las técnicas de deflación [12] y los algoritmos de búsqueda exhaustiva con garantías de completitud [13]. Estos enfoques son especialmente útiles cuando se necesita construir un conjunto de datos que incluya todas las soluciones del sistema para distintos valores de los parámetros.

En un plano distinto, la regresión simbólica [14] persigue descubrir expresiones matemáticas que relacionan variables de entrada con variables de salida sin imponer una forma funcional predefinida. Herramientas recientes como `PySR` [15], `SymbolFit` [16] o `leaf` [17] han automatizado este proceso con creciente eficiencia. Paralelamente, se han explorado estrategias para manejar conjuntos de datos donde coexisten puntos que responden a funciones matemáticas diferentes [18]. Estas últimas resultan especialmente pertinentes cuando se enfrenta un sistema paramétrico cuyas solucio-

nes no pueden ser descritas por una única función, sino que cada una responde a una expresión matemática diferente.

Un intento de conectar el mundo numérico con el simbólico es el presentado en [19], donde se utilizan redes neuronales entrenadas sobre soluciones numéricas de sistemas paramétricos para predecir aproximaciones iniciales que luego se refinan con métodos iterativos. Sin embargo, esta estrategia no proporciona expresiones analíticas explícitas: las redes funcionan como una caja negra que ofrece una aproximación numérica, pero no revela la forma funcional de la solución.

A pesar de estos avances, cuando un sistema no lineal paramétrico admite varias soluciones para una misma tupla de parámetros, ni la regresión simbólica convencional ni las técnicas competitivas existentes ofrecen una vía directa para recuperar todas las dependencias funcionales a partir de un mismo conjunto de datos. A diferencia de un conjunto etiquetado donde se conoce de antemano a qué solución pertenece cada punto, aquí el dato solo indica que un cierto vector de incógnitas satisface el sistema para unos parámetros dados, pero no incluye información que permita distinguir de cuál de las posibles soluciones se trata. Como consecuencia, puntos originados por distintas funciones conviven en el mismo conjunto sin que se pueda separarlos a priori, y el desafío consiste en caracterizar cada una de esas funciones sin conocer cuántas son ni qué forma tienen.

Teniendo en cuenta lo anterior, la pregunta que guía esta investigación es: ¿es posible, mediante una estrategia de regresión simbólica, recuperar expresiones analíticas de los ceros de un sistema de ecuaciones no lineales paramétrico?

El objeto de estudio se sitúa en la intersección del análisis numérico y el aprendizaje automático simbólico. De manera más específica, se aborda la regresión simbólica en el contexto de los sistemas de ecuaciones no lineales paramétricos. El interés se centra en aplicarla a conjuntos de datos generados mediante resolución numérica, con el propósito de separar y descubrir las múltiples funciones solución que en ellos conviven.

El objetivo general de este trabajo es diseñar e implementar un algoritmo que recupere automáticamente las expresiones analíticas de todas las soluciones de un sistema de ecuaciones no lineales paramétricas. Para alcanzarlo, el algoritmo se apoya en una estrategia de regresión simbólica. A partir de este objetivo se desprenden los siguientes objetivos específicos:

- Construir un procedimiento automático que genere un conjunto de entrenamiento formado por tuplas (parámetros, solución), muestreando el espacio de parámetros y resolviendo numéricamente el sistema para cada tupla, de modo que todas las soluciones queden registradas.
- Desarrollar una estrategia de regresión simbólica que, a partir del conjunto de datos completo, descubra secuencialmente las funciones que representan cada

una de las soluciones del sistema, separando de manera automática las distintas dependencias funcionales.

- Validar experimentalmente la metodología sobre sistemas de ecuaciones no lineales con número conocido de soluciones, comparando las expresiones recuperadas con las soluciones exactas cuando sea posible y analizando la precisión de las funciones obtenidas.

La solución propuesta consiste en un algoritmo que alterna etapas de entrenamiento de modelos de regresión simbólica con un paso de filtrado de puntos ya explicados. En cada iteración se entrena un modelo basado en programación genética sobre los datos aún no cubiertos; de entre todas las ecuaciones candidatas que produce el modelo, se selecciona la que consigue ajustar una mayor fracción de puntos y, a continuación, esos puntos se retiran del conjunto de trabajo. El proceso se repite hasta que no quedan puntos suficientes.

Los experimentos realizados sobre sistemas con múltiples soluciones muestran que el algoritmo es capaz de recuperar las expresiones analíticas que dieron origen a los datos, incluso cuando las distintas soluciones se entremezclan en el conjunto de entrenamiento. En los casos estudiados, las funciones fueron descubiertas con una cobertura superior al 95 % de los puntos, y su forma coincidió con las expresiones exactas conocidas.

Durante el desarrollo de este trabajo se empleó un asistente de inteligencia artificial (IA) como herramienta de apoyo en las siguientes tareas: propuesta de estructura para el documento y planificación de los contenidos de cada capítulo, redacción y revisión iterativa de borradores, creación de ejemplos, selección y definición de casos de prueba para los experimentos, generación de soluciones analíticas esperadas para dichos casos, procesamiento de archivos de resultados experimentales, asistencia en la interpretación de pruebas de hipótesis, escritura de código $\text{\LaTeX}$ para tablas y corrección de problemas de formato. El tutor empleó igualmente la IA para asistirle en la redacción de la opinión del tutor.

La estructura del documento es la siguiente. En el Capítulo 1 se presentan los fundamentos de los sistemas de ecuaciones no lineales paramétricos, los métodos numéricos para el cálculo de sus soluciones y los principios de la regresión simbólica basada en programación genética. El Capítulo 2 describe la metodología propuesta: la generación de la malla de parámetros y del conjunto de entrenamiento, la estrategia iterativa de descubrimiento de funciones y los criterios de selección y parada. También se detalla la arquitectura del sistema implementado, incluyendo los módulos de definición de ecuaciones, generación de la malla, resolución numérica, preparación de datos y regresión simbólica. El Capítulo 3 expone los experimentos realizados y el análisis de los resultados. Finalmente, se presentan las conclusiones del trabajo.

Capítulo 1

Preliminares

Este capítulo establece los fundamentos teóricos necesarios para abordar el problema de descubrir expresiones analíticas de los ceros de sistemas de ecuaciones no lineales mediante regresión simbólica. Para ello se presentan los fundamentos conceptuales, las herramientas numéricas y las estrategias algorítmicas que sustentan la metodología propuesta en este trabajo.

En la primera parte del capítulo (1.1) se introduce la notación y las propiedades de los sistemas de ecuaciones no lineales paramétricos, destacando la multiplicidad de soluciones que pueden existir para una misma tupla de parámetros. Se describen los métodos numéricos para el cálculo de ceros de tales sistemas, tanto los métodos iterativos clásicos como el método híbrido de Powell implementado en la biblioteca `SciPy` [20]. Se definen formalmente los conceptos de distancia euclidiana y producto cartesiano, que serán utilizados en la generación del conjunto de datos de entrenamiento.

Posteriormente, en la sección 1.2 se abordan los principios de la regresión simbólica [14], una técnica de aprendizaje automático que permite inferir expresiones algebraicas a partir de datos. Se explican los fundamentos de la programación genética, las funciones de pérdida que guían el proceso evolutivo y los parámetros que controlan el comportamiento del algoritmo.

1.1. Sistemas de ecuaciones no lineales paramétricos

En esta sección se formaliza el concepto de sistema de ecuaciones no lineales dependiente de parámetros, así como las herramientas numéricas y matemáticas necesarias para su estudio.

Un sistema de ecuaciones no lineales paramétrico se expresa como la búsqueda de

vectores $\mathbf{x} \in \mathbb{R}^n$ que satisfacen

$$\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}, \quad (1.1)$$

donde:

- $\mathbf{F} : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^n$ es una función vectorial no lineal,
- $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ es el vector de variables incógnita,
- $\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)^T \in \mathbb{R}^p$ es el vector de parámetros del modelo,
- $\mathbf{0} \in \mathbb{R}^n$ denota el vector nulo.

La notación $\mathbf{F}(\mathbf{x}; \boldsymbol{\theta})$ indica que $\mathbf{F}$ depende explícitamente de las variables $\mathbf{x}$ y de los parámetros $\boldsymbol{\theta}$, estos últimos considerados fijos durante el proceso de resolución.

Ejemplo 1. Considérese el sistema de dos ecuaciones

$$\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \begin{pmatrix} (x_1 - a)(x_2 - ab) \\ x_1 x_2 - ab^2 \end{pmatrix} = \mathbf{0},$$

con $\mathbf{x} = (x_1, x_2)^T \in \mathbb{R}^2$ y parámetros $\boldsymbol{\theta} = (a, b)^T$, $a > 0$, $b > 0$. Este sistema posee dos soluciones reales para cada tupla de parámetros: $\mathbf{x} = (a, b^2)$ y $\mathbf{x} = (b, ab)$. Nótese que la primera componente de la solución (a, b^2) anula el factor $(x_1 - a)$ de la primera ecuación, mientras que en la solución (b, ab) es la segunda componente la que anula el factor $(x_2 - ab)$. Esta multiplicidad de ceros y su dependencia explícita de los parámetros ilustran el tipo de fenómenos que aparecen en sistemas no lineales paramétricos y motivan la necesidad de determinar, para cada valor de los parámetros, todas las raíces que serán utilizadas en la construcción del conjunto de datos de entrenamiento.

La no linealidad de $\mathbf{F}$ implica que, a diferencia de los sistemas lineales, no existe un método directo general para obtener todas las soluciones, y la estructura del conjunto solución puede ser compleja. Por esta razón se recurre a métodos numéricos iterativos, que se describen en la siguiente sección.

1.1.1. Métodos numéricos para el cálculo de ceros

Una vez definido el sistema no lineal paramétrico, es necesario obtener numéricamente sus soluciones para distintas tuplas de parámetros. Dado que, salvo en casos triviales, no es posible despejar analíticamente las raíces de un sistema no lineal arbitrario, se recurre a métodos iterativos que aproximan las soluciones de manera sucesiva. A continuación se describen dos de los métodos más utilizados, que serán empleados en la etapa de generación de datos —esto es, en la construcción del conjunto de entrenamiento que alimentará la regresión simbólica—.

Para sistemas no lineales generales, la obtención de soluciones explícitas mediante métodos simbólicos no es factible, ya que la mayoría de estos sistemas carecen de soluciones expresables en forma cerrada [9]. En su lugar, se emplean métodos numéricos iterativos que, partiendo de una aproximación inicial $\mathbf{x}^{(0)}$, generan una sucesión $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$ que converge a una solución $\mathbf{x}^*$, siempre que la aproximación inicial esté suficientemente cerca de dicha solución y que $\mathbf{F}$ sea diferenciable con jacobiano no singular en $\mathbf{x}^*$ [10].

El método de Newton-Raphson para sistemas no lineales [9] se define mediante la iteración

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - [\mathbf{J}_{\mathbf{F}}(\mathbf{x}^{(k)})]^{-1} \mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta}), \quad (1.2)$$

donde:

- $\mathbf{x}^{(k)}$ es la aproximación en la iteración k ,
- $\mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta})$ es el valor de $\mathbf{F}$ evaluado en dicha aproximación,
- $\mathbf{J}_{\mathbf{F}}(\mathbf{x})$ es la matriz jacobiana de $\mathbf{F}$ respecto de $\mathbf{x}$, de dimensión $n \times n$, con entradas $(\mathbf{J}_{\mathbf{F}})_{ij} = \frac{\partial F_i}{\partial x_j}$.

Si $\mathbf{F}$ es diferenciable y $\mathbf{J}_{\mathbf{F}}$ es no singular en una solución $\mathbf{x}^*$, entonces existe un entorno de $\mathbf{x}^*$ donde el método converge cuadráticamente, es decir, $\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \leq C\|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2$ para alguna constante $C > 0$ [10].

Los métodos cuasi-Newton [10] reducen el costo computacional al evitar el cálculo exacto del jacobiano en cada iteración. En lugar de ello, construyen una aproximación $\mathbf{B}_k$ de la matriz jacobiana que se actualiza con información de los gradientes evaluados en iteraciones previas. Las actualizaciones típicas son de rango uno o dos, lo que significa que modifican la matriz anterior sumando una matriz de rango uno o dos, respectivamente. La iteración toma la forma

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{B}_k^{-1} \mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta}),$$

donde $\mathbf{B}_k$ se actualiza mediante fórmulas como la de Broyden [21].

La actualización de Broyden está definida por

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{(\mathbf{y}_k - \mathbf{B}_k \mathbf{s}_k) \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{s}_k},$$

donde $\mathbf{s}_k = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$ y $\mathbf{y}_k = \mathbf{F}(\mathbf{x}^{(k+1)}) - \mathbf{F}(\mathbf{x}^{(k)})$.

Estos métodos alcanzan una convergencia superlineal [10]: la tasa de reducción del error es más rápida que la lineal pero sin llegar a la velocidad cuadrática del método de Newton completo.

En este trabajo se utilizará un método híbrido, como el implementado en la biblioteca SciPy [20], que combina una estrategia cuasi-Newton con búsqueda de paso para mejorar la robustez. La búsqueda de paso consiste en ajustar la longitud de la corrección que se aplica en cada iteración para garantizar una disminución suficiente de la función de mérito, mejorando así la convergencia global del método. A partir de este procedimiento se generará el conjunto de datos de entrenamiento para la regresión simbólica, el cual está formado por múltiples tuplas $(\boldsymbol{\theta}, \mathbf{x}^*)$, donde $\mathbf{x}^*$ es una raíz obtenida numéricamente para el parámetro $\boldsymbol{\theta}$.

Las soluciones obtenidas en esta fase serán posteriormente filtradas para eliminar repeticiones, lo que motiva el estudio de la distancia euclidiana en la siguiente sección.

1.1.2. Distancia euclidiana

En la etapa de resolución numérica, dos raíces obtenidas a partir de puntos iniciales distintos pueden corresponder al mismo cero del sistema, diferenciándose únicamente por errores de redondeo o por las tolerancias de convergencia de los métodos iterativos. Para decidir si dos soluciones deben considerarse iguales, se utiliza la distancia euclidiana.

Definición 1.1 (Distancia euclidiana) Sean $\mathbf{u} = (u_1, \dots, u_n)^T \in \mathbb{R}^n$ y $\mathbf{v} = (v_1, \dots, v_n)^T \in \mathbb{R}^n$ dos vectores. La **distancia euclidiana** entre $\mathbf{u}$ y $\mathbf{v}$ se define como la norma euclidiana de su diferencia:

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n (u_i - v_i)^2}. \quad (1.3)$$

La distancia euclidiana constituye una métrica sobre $\mathbb{R}^n$. En este trabajo, dos soluciones $\mathbf{x}$ y $\mathbf{x}'$ se consideran numéricamente equivalentes si su distancia euclidiana es menor que una tolerancia prefijada $\tau > 0$, lo que típicamente ocurre cuando ambas corresponden al mismo cero del sistema pero difieren debido a la acumulación de errores numéricos.

En particular, la **norma euclidiana** de un vector $\mathbf{v} \in \mathbb{R}^n$ se define como su distancia al vector nulo, es decir,

$$\|\mathbf{v}\|_2 = d(\mathbf{v}, \mathbf{0}) = \sqrt{\sum_{i=1}^n v_i^2}. \quad (1.4)$$

Esta cantidad resulta útil para medir qué tan cerca está un punto de ser solución del sistema: si $\|\mathbf{F}(\mathbf{x}; \boldsymbol{\theta})\|_2$ es pequeño, entonces $\mathbf{x}$ está próximo a anular todas las ecuaciones.

Una vez identificadas las soluciones distintas para cada tupla de parámetros, es necesario organizar sistemáticamente los valores de dichos parámetros sobre los que se evaluará el sistema. Esta organización se realiza mediante el producto cartesiano, que se define a continuación.

1.1.3. Producto cartesiano

Para construir el conjunto de datos de entrenamiento se debe evaluar el sistema no lineal en una colección representativa de valores de los parámetros. Con este fin se selecciona, para cada parámetro θ_i , un conjunto finito de valores dentro de su rango admisible; el producto cartesiano de estos conjuntos proporciona todas las combinaciones posibles de parámetros que se utilizarán en la etapa de resolución numérica.

Definición 1.2 (Producto cartesiano) Sean $A_1, A_2, \dots, A_m$ conjuntos arbitrarios. El **producto cartesiano** de estos conjuntos se define como el conjunto de todas las m -tuplas ordenadas $(a_1, a_2, \dots, a_m)$ tales que $a_i \in A_i$ para cada $i = 1, \dots, m$. Formalmente:

$$A_1 \times A_2 \times \dots \times A_m = \{(a_1, a_2, \dots, a_m) \mid a_i \in A_i, i = 1, \dots, m\}. \quad (1.5)$$

Si cada A_i es un conjunto finito, el cardinal del producto cartesiano es el producto de los cardinales: $|A_1 \times \dots \times A_m| = \prod_{i=1}^m |A_i|$. En el contexto de este trabajo, cada A_i es un conjunto de valores del parámetro θ_i , típicamente elegidos de manera equiespaciada dentro de un intervalo $[\theta_i^{\min}, \theta_i^{\max}]$. El producto cartesiano de estos conjuntos genera todas las tuplas $\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)$ que serán utilizadas como entrada del sistema durante la generación de datos, garantizando una cobertura uniforme de la región de interés en el espacio de parámetros.

Ejemplo 2 (proceso de generación de datos). Considérese un sistema con dos parámetros a y b , cada uno de los cuales debe tomar valores en el intervalo $[0.1, 5]$. Se discretiza el rango de cada parámetro mediante 50 puntos equiespaciados, obteniéndose dos conjuntos finitos Θ_a y Θ_b . El producto cartesiano $\Theta_a \times \Theta_b$ contiene 2500 tuplas distintas. Para cada una de estas tuplas se ejecutará el método numérico desde varios puntos iniciales aleatorios, generando así el conjunto de pares $(\boldsymbol{\theta}, \mathbf{x})$ que constituye el dato de entrada de la regresión simbólica. Este mismo procedimiento se generaliza de manera directa a cualquier número de parámetros.

Una vez definidas las herramientas para generar los valores de los parámetros y para identificar las raíces del sistema, se dispone del marco necesario para producir el conjunto de datos con el que se alimentará el algoritmo de regresión simbólica, cuyos fundamentos se exponen en la siguiente sección.

1.2. Regresión simbólica

La regresión simbólica es la técnica central sobre la que se apoya la metodología de este trabajo. A continuación se presentan sus fundamentos teóricos y los elementos que la componen.

1.2.1. Concepto y objetivos

La regresión simbólica es una técnica de aprendizaje automático que descubre, a partir de un conjunto de datos observados, una expresión matemática cerrada. Dicha expresión describe la relación funcional entre las variables de entrada y la variable de salida [14]. A diferencia de la regresión tradicional, donde la estructura del modelo se fija de antemano y solo se ajustan los parámetros, la regresión simbólica explora simultáneamente el espacio de formas funcionales y de parámetros numéricos [22].

Existen diversas implementaciones de regresión simbólica, muchas de las cuales se basan en programación genética, enfoque que se describe en la siguiente subsección. Una de las más utilizadas es **PySR** [15], una biblioteca de código abierto que combina un motor evolutivo en Julia con una interfaz en Python, ofreciendo operadores matemáticos protegidos —versiones de funciones como la raíz cuadrada o la división que evitan valores indefinidos (NaN o infinitos) durante la búsqueda— para garantizar la estabilidad numérica.

A continuación se formaliza el problema de regresión simbólica desde un punto de vista matemático.

Dado un conjunto de datos $\mathcal{D} = \{(\mathbf{u}_i, v_i)\}_{i=1}^N$ donde:

- $\mathbf{u}_i \in \mathbb{R}^d$ es el vector de variables de entrada del i -ésimo punto del conjunto de datos,
- $v_i \in \mathbb{R}$ es el valor observado de la variable de salida para dicho punto,

se busca una función $f : \mathbb{R}^d \rightarrow \mathbb{R}$ dentro de un espacio de funciones $\mathcal{F}$ que minimice una función de pérdida $\mathcal{L}$, la cual mide la discrepancia entre las predicciones del modelo y los valores observados:

$$f^* = \arg \min_{f \in \mathcal{F}} \mathcal{L}(f; \mathcal{D}). \quad (1.6)$$

El espacio $\mathcal{F}$ está compuesto por todas las funciones que pueden construirse combinando un conjunto de operadores matemáticos básicos (suma, resta, multiplicación, división, logaritmos, etc.) y constantes numéricas. La tarea de encontrar la función óptima es intrínsecamente combinatoria: el número de expresiones que pueden formarse crece de manera exponencial con la cantidad de operadores y variables disponibles,

y se ha demostrado que el problema de regresión simbólica es NP-duro [23]. Por esta razón se requieren métodos de búsqueda heurística, como los algoritmos evolutivos.

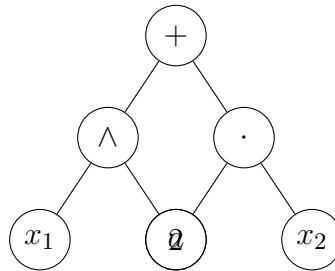
La implementación más extendida de estos algoritmos para regresión simbólica es la programación genética, cuyos fundamentos se exponen a continuación.

1.2.2. Fundamentos de programación genética

Una de las estrategias más utilizadas para la regresión simbólica es la **programación genética**, una rama de los algoritmos evolutivos inspirada en la selección natural [14]. Existen también enfoques basados en recocido simulado [24], gramáticas estocásticas [25] y redes neuronales [26, 27], aunque la programación genética sigue siendo una de las opciones más extendidas por su flexibilidad para explorar espacios de expresiones algebraicas.

En este paradigma, cada posible expresión matemática se codifica como un **árbol sintáctico**: una estructura jerárquica en la que los nodos internos contienen operadores matemáticos (suma, multiplicación, seno, etc.) y las hojas contienen variables de entrada o constantes numéricas. La figura resultante tiene forma de árbol invertido, donde la raíz es el operador principal y las ramas conducen a los operandos.

Ejemplo 3 (árbol sintáctico). La expresión $x_1^2 + a \cdot x_2$ se representa mediante un árbol cuya raíz es el operador $+$. El subárbol izquierdo codifica x_1^2 con el operador $\wedge$ y las hojas x_1 y 2 ; el subárbol derecho codifica $a \cdot x_2$ con el operador $\cdot$ y las hojas a y x_2 . Esta representación permite que los operadores genéticos manipulen directamente la estructura de las expresiones.



La programación genética trabaja con una población de individuos (árboles) que se modifica a lo largo de generaciones —ciclos iterativos del proceso evolutivo— mediante los siguientes operadores:

- **Selección:** Los individuos que producen predicciones más cercanas a los valores reales tienen mayor probabilidad de pasar a la siguiente generación.
- **Cruce:** Dos árboles padres intercambian subárboles para generar descendientes que combinan características de ambos.

- **Mutación:** Se reemplaza aleatoriamente un nodo o subárbol por otro nuevo, introduciendo diversidad en la población.
- **Elitismo:** Una fracción de los mejores individuos de cada generación se preserva sin cambios en la siguiente, evitando así que se pierdan las soluciones más prometedoras. Este mecanismo es opcional y su intensidad depende del problema [14].

La repetición de estos operadores a lo largo de las generaciones permite explorar el espacio de posibles expresiones algebraicas, guiando la búsqueda hacia funciones que se ajustan cada vez mejor a los datos [14].

La calidad de cada individuo se mide mediante una función de pérdida, cuyas variantes más relevantes para este trabajo se presentan a continuación.

1.2.3. Funciones de pérdida

La **función de pérdida** —también llamada función de aptitud en el contexto de los algoritmos evolutivos— es el criterio que se utiliza para evaluar cuantitativamente qué tan buena es una expresión candidata producida por la regresión simbólica. Esta función mide la discrepancia entre las predicciones de la expresión simbólica y los valores observados en el conjunto de datos. Una función de pérdida común, y la que se emplea como referencia en este trabajo, es el error cuadrático medio.

Error cuadrático medio. Para un conjunto de datos $\{(\mathbf{u}_i, v_i)\}_{i=1}^N$ con $\mathbf{u}_i \in \mathbb{R}^d$ y $v_i \in \mathbb{R}$, y una función $f: \mathbb{R}^d \rightarrow \mathbb{R}$, el error cuadrático medio (MSE, por sus siglas en inglés) se define como

$$\text{MSE}(f) = \frac{1}{N} \sum_{i=1}^N (v_i - f(\mathbf{u}_i))^2, \quad (1.7)$$

donde:

- N es el número total de puntos en el conjunto de datos,
- v_i es el valor observado para el i -ésimo punto,
- $f(\mathbf{u}_i)$ es el valor predicho por la expresión f para el vector de entrada $\mathbf{u}_i$.

El MSE penaliza cuadráticamente las desviaciones, por lo que es sensible a valores atípicos [28]. En el contexto de la regresión simbólica, es una de las funciones de pérdida más utilizadas debido a su simplicidad y a que proporciona una señal suave para guiar el proceso de búsqueda [29].

La efectividad del proceso evolutivo no solo depende de la función de pérdida elegida, sino también de la configuración del algoritmo. A continuación se describen los parámetros que controlan el comportamiento de la programación genética aplicada a la regresión simbólica.

1.2.4. Parámetros del algoritmo de regresión simbólica

El comportamiento de la regresión simbólica basada en programación genética depende de un conjunto de parámetros que influyen en la calidad de las soluciones, el tiempo de cómputo y la interpretabilidad de los resultados [15]. A continuación se describen los más relevantes, agrupados según el aspecto del algoritmo que controlan.

Parámetros de la población.

- **Tamaño de la población (P):** Número de individuos que coexisten en cada generación. Una población grande aumenta la diversidad genética y la probabilidad de encontrar soluciones de alta calidad, a cambio de un mayor tiempo de cómputo y consumo de memoria.
- **Número de generaciones (G):** Cantidad máxima de iteraciones. Un número elevado permite explorar más a fondo el espacio de búsqueda, aunque también incrementa el costo computacional.
- **Número de poblaciones independientes:** En algunas implementaciones se utiliza un modelo de islas, que consiste en dividir la población total en varias subpoblaciones que evolucionan en paralelo. Periódicamente, un pequeño número de individuos migra entre las islas, lo que ayuda a mantener la diversidad genética y evita la convergencia prematura [30].

Parámetros de control de la complejidad. Además de la precisión, interesa obtener expresiones sencillas que sean fáciles de interpretar. Los siguientes parámetros regulan el equilibrio entre ajuste a los datos y simplicidad de las fórmulas.

- **Coeficiente de parsimonia (α):** Penalización añadida a la función de pérdida, proporcional al tamaño del árbol (número de nodos). La pérdida efectiva se calcula como

$$\mathcal{L}_{\text{eff}}(f) = \text{MSE}(f) + \alpha \cdot \text{tamaño}(f).$$

Un valor mayor de α favorece expresiones más simples.

- **Tamaño máximo del árbol ($S_{\text{máx}}$):** Límite en el número de nodos que puede tener un individuo. Evita la explosión de la complejidad y reduce el espacio de búsqueda.

Probabilidades de operadores genéticos. La composición de cada nueva generación está determinada por las siguientes probabilidades, que deciden qué operador se aplica a cada individuo.

- **Probabilidad de cruce** (p_c): Fracción de la nueva generación generada mediante cruce de padres seleccionados.
- **Probabilidad de mutación** (p_m): Fracción de individuos creados mediante modificaciones aleatorias de los árboles existentes.
- **Probabilidad de reproducción** (p_r): Fracción de individuos que pasan directamente a la siguiente generación sin cambios (elitismo), con $p_r = 1 - p_c - p_m$.

En síntesis, el proceso de regresión simbólica mediante programación genética comienza con una población inicial de árboles generados aleatoriamente. Cada árbol es evaluado con la función de pérdida y se seleccionan los mejores para la siguiente generación. Mediante cruce y mutación se crean nuevos individuos, mientras que el elitismo preserva las mejores soluciones. Este ciclo se repite durante un número prefijado de generaciones o hasta que se alcanza una calidad suficiente, dando como resultado una expresión analítica que aproxima los datos.

Para evaluar estadísticamente los resultados experimentales se utiliza la prueba *t* pareada, cuyos fundamentos se resumen a continuación.

1.3. Prueba *t* pareada

Durante la experimentación se comparan distintas funciones de pérdida evaluadas sobre los mismos casos de prueba. Para decidir si una de ellas ofrece un desempeño significativamente mejor que otra se emplea la **prueba *t* de Student para muestras pareadas** [31]. Esta prueba contrasta la hipótesis nula de que la diferencia media entre ambos métodos es igual a un valor dado (habitualmente cero) frente a la alternativa de que dicha diferencia es mayor o menor, según la dirección de la mejora que se quiere detectar.

Si denotamos por $d_i = x_i - y_i$ la diferencia entre los resultados de dos métodos para el i -ésimo caso, el estadístico de prueba se calcula como

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}, \quad \bar{d} = \frac{1}{n} \sum_{i=1}^n d_i, \quad s_d = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (d_i - \bar{d})^2},$$

donde n es el número de pares. Bajo los supuestos habituales de independencia y normalidad aproximada de las diferencias, el estadístico t sigue una distribución *t* de Student, la cual es simétrica y con colas más pesadas que la distribución normal estándar, especialmente para tamaños muestrales pequeños. El parámetro que determina su forma es el número de **grados de libertad**, $\nu = n - 1$, que refleja la cantidad de datos disponibles para estimar la variabilidad de las diferencias. Conforme ν crece, la distribución *t* se aproxima a la normal.

A partir del valor observado de t y de los grados de libertad se obtiene el **p -valor**, que indica la probabilidad de obtener un resultado al menos tan extremo como el observado si la hipótesis nula fuera cierta. Un p -valor inferior a un nivel de significación prefijado (en este trabajo $\alpha = 0.05$) conduce a rechazar la hipótesis nula en favor de la alternativa.

En los análisis del Capítulo 3 se aplica la prueba t pareada para comparar tanto la tasa de éxito como el tiempo de ejecución de las distintas funciones de pérdida, indicando en cada caso el valor del estadístico t y el correspondiente p -valor.

Con esta base, en el siguiente capítulo se desarrollará la metodología propuesta, que integra los conceptos aquí descritos para construir y entrenar modelos de regresión simbólica capaces de recuperar las expresiones analíticas de las raíces a partir de datos numéricos.

Capítulo 2

Propuesta de solución

En este capítulo se desarrolla la metodología propuesta para recuperar las expresiones analíticas de los ceros de sistemas de ecuaciones no lineales paramétricos. A diferencia de una resolución puramente numérica, que proporciona valores puntuales para cada tupla de parámetros, se busca descubrir las funciones que relacionan dichos parámetros con las raíces del sistema. Este propósito requiere resolver dos tareas: obtener un conjunto de datos que represente adecuadamente el comportamiento del sistema e inferir, a partir de esos datos, las expresiones analíticas que subyacen.

En la Sección 2.2 se aborda la primera tarea. Allí se explica cómo se construye el conjunto de entrenamiento a partir de una malla de parámetros, es decir, un conjunto finito de tuplas obtenidas al elegir puntos equiespaciados para cada parámetro y combinarlos mediante producto cartesiano. Sobre esa malla se resuelve el sistema numéricamente y se filtran las soluciones duplicadas. La segunda tarea se aborda en la Sección 2.3, donde se presenta la estrategia de regresión simbólica multidimensional. Esta estrategia emplea una búsqueda iterativa que descubre progresivamente las distintas expresiones que componen la solución, validando cada una frente a los datos restantes. Como parte de la estrategia se describen tres variantes de función de pérdida —error cuadrático medio, conteo de coincidencias y sigmoideal—, cuya comparación experimental permitirá elegir la más adecuada. El capítulo concluye con el pseudocódigo completo del algoritmo (Sección 2.4).

2.1. Planteamiento general del problema

El objetivo central de este trabajo es encontrar las expresiones analíticas que describen todas las soluciones del sistema

$$\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}, \tag{2.1}$$

definido en la Sección 1.1.

Para un valor fijo de los parámetros θ , el sistema puede tener más de una solución. En lugar de calcular numéricamente esas soluciones para cada θ por separado, lo que se persigue es obtener una aproximación simbólica que, para cualquier θ dentro de una región de interés $\Omega \subset \mathbb{R}^p$, permita estimar cada una de las soluciones sin necesidad de repetir el proceso numérico. No se exige una expresión exacta válida para todos los valores imaginables de los parámetros, sino una descripción funcional que reproduzca fielmente el comportamiento del sistema en la región donde se han tomado los datos.

Como estas soluciones forman naturalmente grupos que varían de manera continua con los parámetros, las organizamos en una lista indexada:

$$\mathbf{x} = \mathbf{g}^{(1)}(\theta), \quad \mathbf{x} = \mathbf{g}^{(2)}(\theta), \quad \dots$$

Aquí, $\mathbf{g}^{(k)}$ es una función que toma parámetros y devuelve un vector con los valores de todas las variables de la solución k -ésima. El superíndice k simplemente distingue las distintas soluciones que pueden coexistir para una misma θ . Con este planteamiento, el problema queda reducido a descubrir automáticamente, a partir de datos, las expresiones matemáticas de esas funciones $\mathbf{g}^{(k)}$.

Mientras que los métodos numéricos tradicionales proporcionan soluciones puntuales para valores fijos de los parámetros [9], el método que se propone busca describir cómo varía cada solución en función de los parámetros. Esto permite evaluar las soluciones para nuevos valores de los parámetros sin necesidad de repetir el proceso numérico, y facilita el análisis cuantitativo del sistema al hacer explícita la relación entre parámetros y raíces.

La estrategia para lograr este objetivo se apoya en dos pilares: la generación sistemática de un conjunto de datos representativo del sistema y la aplicación de regresión simbólica sobre esos datos. La primera de estas tareas se aborda en la sección siguiente.

2.2. Generación de datos de entrenamiento

La primera etapa del método consiste en construir un conjunto de datos que relacione los parámetros del sistema con sus soluciones. Este conjunto se obtiene muestreando el espacio de parámetros, resolviendo el sistema para cada tupla y filtrando las soluciones duplicadas.

2.2.1. Muestreo del espacio de parámetros

Para cada parámetro θ_i se elige un conjunto finito Θ_i de valores equiespaciados dentro de un intervalo $[\theta_i^{\min}, \theta_i^{\max}]$. El producto cartesiano

$$\mathcal{G} = \Theta_1 \times \Theta_2 \times \dots \times \Theta_p$$

genera todas las combinaciones posibles de parámetros. Esta construcción se apoya en la definición de producto cartesiano presentada en la Sección 1.1 y produce una cuadrícula regular que cubre uniformemente la región de interés.

Ejemplo 2. Para un sistema con un solo parámetro $a \in [0.1, 5]$ y 50 puntos, se tiene $\Theta_a = \{0.1, 0.2, \dots, 5.0\}$ y la cuadrícula es $\mathcal{G} = \Theta_a$. Si el sistema tuviera dos parámetros a y b en el mismo intervalo con 50 puntos cada uno, la cuadrícula $\mathcal{G} = \Theta_a \times \Theta_b$ contendría 2500 tuplas.

2.2.2. Resolución numérica del sistema

Para cada tupla $\theta \in \mathcal{G}$ se resuelve el sistema $\mathbf{F}(\mathbf{x}; \theta) = \mathbf{0}$ mediante un método numérico iterativo. Puesto que pueden existir varias soluciones reales para un mismo valor de los parámetros, se ejecuta el método desde múltiples puntos iniciales aleatorios, cada uno de ellos generado eligiendo cada coordenada x_i dentro de un intervalo predefinido $[x_i^{\min}, x_i^{\max}]$.

Una ejecución del método numérico se considera exitosa si converge y el residuo $\|\mathbf{F}(\mathbf{x}; \theta)\|_2$ está por debajo de una tolerancia preestablecida. Cuando esto ocurre, el vector $\mathbf{x}$ obtenido se agrega a la lista de soluciones encontradas para la tupla θ . Los valores concretos del número de intentos, los intervalos de búsqueda y las tolerancias se fijan durante la experimentación (Capítulo 3).

Una vez calculadas las soluciones para cada tupla de parámetros, el paso siguiente es eliminar aquellas que aparecen repetidas debido a que distintos puntos iniciales convergieron al mismo cero.

2.2.3. Filtrado de soluciones duplicadas

Distintos puntos iniciales pueden conducir a la misma solución real. Para evitar que una misma raíz aparezca repetida, se aplica un filtro basado en la distancia euclidiana (Sección 1.1): dos vectores solución $\mathbf{u}$ y $\mathbf{v}$ se consideran idénticos si $\|\mathbf{u} - \mathbf{v}\|_2 < \tau$, donde τ es una tolerancia que depende de la escala del problema y de la precisión numérica.

Ejemplo 3. Supóngase que para un cierto θ se ejecuta el método desde tres puntos iniciales y se obtienen los vectores $\mathbf{u} = (1.0002, 2.0001)^T$, $\mathbf{v} = (0.9998, 1.9999)^T$ y $\mathbf{w} = (3.1415, 2.7182)^T$. Con una tolerancia $\tau = 10^{-3}$, los vectores $\mathbf{u}$ y $\mathbf{v}$ se consideran la misma solución (su distancia es menor que τ), mientras que $\mathbf{w}$ es una solución distinta.

Tras este proceso se dispone de un conjunto de pares $(\theta, \mathbf{x}^*)$, donde cada $\mathbf{x}^*$ es una de las soluciones reales del sistema para los parámetros θ . Es importante notar que, para una misma tupla θ , pueden existir varios pares en el conjunto de datos, cada uno con un vector $\mathbf{x}^*$ distinto. En otras palabras, cuando el sistema admite más

de una solución para un mismo valor de los parámetros, la tabla de datos contiene múltiples filas con idéntico θ y diferente $\mathbf{x}^*$. Este conjunto constituye la entrada para la etapa de regresión simbólica que se describe en la sección siguiente.

2.3. Estrategia de regresión simbólica

Una vez construido el conjunto de datos —en el que cada tupla de parámetros puede aparecer en varias filas, cada una con un vector solución $\mathbf{x}$ distinto—, el paso siguiente es extraer, a partir de esos datos, las expresiones analíticas subyacentes.

La dificultad principal para aplicar regresión simbólica a estos datos es doble. Por un lado, el conjunto contiene múltiples soluciones para una misma tupla de parámetros; por otro, cada solución está formada por varias variables interdependientes. Abordar directamente el problema como un ajuste vectorial no es viable, porque el algoritmo tendería a predecir valores intermedios que no corresponden a ninguna solución real. En su lugar, se sigue una estrategia en tres fases. Primero, como se describe en la Sección 2.3.1, se descompone el problema tratando cada coordenada de la solución de forma independiente y se elige una de ellas para guiar la separación de las distintas soluciones. A continuación, mediante un proceso iterativo que se presenta en la Sección 2.3.2, se identifican secuencialmente las distintas funciones solución para la coordenada elegida y, a partir de ellas, se obtienen expresiones para las demás variables de la misma solución, como se explica en la Sección 2.3.3. Finalmente, cada solución completa se valida comprobando que satisface el sistema original dentro de una tolerancia preestablecida, según el procedimiento que se detalla en la Sección 2.3.4.

En los apartados que siguen se desarrolla cada una de estas fases.

2.3.1. Separación por coordenadas

El conjunto de datos generado en la etapa anterior asocia cada tupla de parámetros θ con una o varias soluciones del sistema: cuando hay más de una solución para el mismo θ , se añaden varias filas a la tabla de datos, una por cada solución. Esta característica impide abordar el problema como un simple ajuste funcional: si para un mismo θ existen varios valores de $\mathbf{x}$, cualquier función que intente predecir $\mathbf{x}$ a partir de θ estará mal definida, pues a una misma entrada le corresponderían varias salidas. Un modelo de regresión entrenado directamente sobre estos datos tendería a producir un valor intermedio entre las distintas soluciones, que no corresponde a ninguna de ellas.

Para evitar tener que construir una función vectorial que asigne directamente θ al vector completo $\mathbf{x}$, se trata cada coordenada por separado. Así, el objetivo pasa a

ser encontrar n funciones escalares

$$x_k = g_k(\boldsymbol{\theta}), \quad k = 1, \dots, n,$$

todas ellas definidas sobre el mismo espacio de parámetros. La notación g_k representa la función que describe cómo la k -ésima coordenada de la solución depende de los parámetros. Esta separación por coordenadas reduce el problema a hallar expresiones algebraicas para cada variable de forma independiente, pero no resuelve por sí sola la dificultad de la multiplicidad de soluciones: incluso considerando una única coordenada, los datos de esa columna de la tabla pueden contener varios valores para un mismo $\boldsymbol{\theta}$, ya que proceden de soluciones distintas.

Ejemplo 4. Considérese el sistema definido por $x_1^2 - x_2 = 0$ y $x_1 + x_2 = 6$ con un parámetro $a \equiv 2$. Para este valor, el sistema admite dos soluciones: $(x_1, x_2) = (2, 4)$ y $(x_1, x_2) = (3, 9)$. Las filas correspondientes de la tabla de datos serían:

a	x_1	x_2
2	2	4
2	3	9
$\vdots$	$\vdots$	$\vdots$

En la columna de x_1 aparecen dos valores (2 y 3) asociados al mismo $a = 2$, lo que impide aplicar directamente regresión simbólica para predecir x_1 a partir de a .

Para poder separar estos valores, se elige una de las coordenadas como guía para agrupar los puntos. Sin pérdida de generalidad, se toma x_1 como punto de partida. La idea es la siguiente: una vez que se identifiquen las distintas funciones que describen a x_1 y se asigne cada fila de la tabla a una de ellas, los valores de las demás coordenadas quedarán automáticamente asociados a cada grupo, porque en los datos originales cada fila contiene el vector completo $\mathbf{x}$. Así, para un mismo $\boldsymbol{\theta}$, el valor de cada x_k pertenece a la misma solución que el valor de x_1 .

De esta manera, el proceso se organiza en dos etapas. Primero se aplica regresión simbólica sobre los datos de x_1 para descubrir, mediante un procedimiento iterativo, cuántas expresiones diferentes para x_1 están presentes en los datos y a qué subconjunto de filas —es decir, el conjunto de filas de la tabla cuyos valores de x_1 una misma expresión aproxima bien— corresponde cada una. A continuación, se obtienen expresiones para las coordenadas restantes aprovechando los grupos ya formados.

Nótese que este procedimiento asume que, una vez fijada la expresión para x_1 , las demás coordenadas quedan determinadas sin ambigüedad dentro de cada grupo. Si las soluciones no fueran separables por coordenadas —es decir, si dos soluciones distintas compartieran la misma expresión para x_1 pero difirieran en x_2 — sería necesario un paso adicional de verificación de compatibilidad entre coordenadas. Esta verificación

se realiza durante la validación final de la solución completa, que se describe en la Sección 2.3.4.

Ejemplo (organización tras el proceso). Para el sistema del Ejemplo 4, una vez identificadas dos funciones para x_1 , la tabla se reorganiza en dos grupos:

Grupo	a	x_1	x_2
1	2	2	4
1	4	4	16
$\vdots$	$\vdots$	$\vdots$	$\vdots$
2	2	3	9
2	5	5	25
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Los detalles de cada etapa se exponen en las secciones siguientes.

2.3.2. Búsqueda iterativa sobre x_1

Una vez organizados los datos, se aplica el algoritmo de regresión simbólica descrito en la Sección 1.2 a la primera coordenada. Para ello, de la tabla completa —que tiene columnas para cada parámetro y cada variable— se extrae una sub-tabla que contiene únicamente las columnas de los parámetros θ y la columna de x_1 . El objetivo de esta etapa es identificar, de manera iterativa, las distintas funciones que describen la primera coordenada de las soluciones.

El proceso iterativo es necesario porque en los datos de x_1 conviven puntos que responden a funciones diferentes, y la regresión simbólica estándar solo puede ajustar una de ellas en cada ejecución. El algoritmo comienza con el conjunto completo de puntos de la sub-tabla y, en cada iteración, entrena un modelo de regresión simbólica que busca una expresión $g_1^{(j)}(\theta)$ tal que

$$|x_{1,i} - g_1^{(j)}(\theta_i)| < \varepsilon$$

para la mayor cantidad posible de puntos, donde ε es una tolerancia preestablecida y el subíndice i indexa las filas de la tabla. El superíndice (j) indica que se trata de la j -ésima solución descubierta para x_1 .

Los puntos que satisfacen la desigualdad anterior para la expresión $g_1^{(j)}$ se retiran del conjunto de datos, y la iteración siguiente trabaja exclusivamente con los puntos restantes. Este procedimiento se repite hasta que el número de puntos sin explicar es menor que un umbral mínimo, momento en el cual se considera que todas las soluciones relevantes han sido identificadas.

Ejemplo 5. Retomando el Ejemplo 4, la sub-tabla de x_1 contiene los siguientes datos:

a	x_1
2	2
2	3
4	4
5	5
5	6
$\vdots$	$\vdots$

En la primera iteración, el modelo de regresión simbólica descubre la expresión $g_1^{(1)}(a) = a$, que ajusta los puntos donde $x_1 = a$ (por ejemplo, $a = 2 \rightarrow 2$, $a = 4 \rightarrow 4$, $a = 5 \rightarrow 5$). Estos puntos se retiran. En la segunda iteración, trabajando solo con los puntos restantes, el modelo encuentra $g_1^{(2)}(a) = a + 1$, que ajusta los puntos donde $x_1 = a + 1$ (por ejemplo, $a = 2 \rightarrow 3$, $a = 5 \rightarrow 6$). Tras esta segunda iteración no quedarían puntos, y el algoritmo concluiría que existen dos expresiones para x_1 : $g_1^{(1)}(a) = a$ y $g_1^{(2)}(a) = a + 1$.

Los puntos que permanecen sin explicar al finalizar el proceso pueden deberse a soluciones cuya expresión es demasiado compleja para ser descubierta con los recursos de cómputo disponibles, a errores numéricos en la resolución del sistema, o a que el número de iteraciones fue insuficiente. En el capítulo de experimentación se analiza este fenómeno con los datos obtenidos.

Cada expresión $g_1^{(j)}$ queda asociada al subconjunto de parámetros para los que la predicción es correcta dentro de la tolerancia ε . Formalmente, se define

$$\mathcal{S}^{(j)} = \{\theta \in \mathcal{G} \mid |x_1 - g_1^{(j)}(\theta)| < \varepsilon\},$$

es decir, el conjunto de tuplas de parámetros para las cuales la expresión $g_1^{(j)}$ ajusta el valor observado de x_1 .

Ejemplo 6. En el Ejemplo 5, la expresión $g_1^{(1)}(a) = a$ estaría asociada al subconjunto $\mathcal{S}^{(1)} = \{2, 4, 5, \dots\}$ (los valores de a para los que $x_1 = a$), mientras que $g_1^{(2)}(a) = a + 1$ lo estaría a $\mathcal{S}^{(2)} = \{2, 5, \dots\}$ (los valores de a para los que $x_1 = a + 1$).

$\mathcal{S}^{(1)}$	$\mathcal{S}^{(2)}$
2	2
4	5
5	$\vdots$
$\vdots$	

Puesto que los valores de x_1 para una misma tupla de parámetros pueden provenir de soluciones distintas, cada una de estas funciones identifica una solución del sistema.

Al finalizar esta etapa se cuenta con una lista de funciones para x_1 , cada una con su subconjunto de parámetros asociado, lo que proporciona la base para obtener expresiones para las coordenadas restantes, como se describe en la sección siguiente.

2.3.3. Extensión a las demás coordenadas

Una vez que la búsqueda iterativa ha proporcionado una lista de expresiones $g_1^{(1)}, g_1^{(2)}, \dots$ para la primera coordenada, junto con los subconjuntos de parámetros $\mathcal{S}^{(1)}, \mathcal{S}^{(2)}, \dots$ que cada una explica, el paso siguiente es completar la descripción de cada solución del sistema. Para ello se obtienen expresiones simbólicas para las coordenadas restantes $x_2, x_3, \dots, x_n$.

Para una solución particular, identificada por el índice j , se dispone del subconjunto $\mathcal{S}^{(j)}$. Para cada $\theta \in \mathcal{S}^{(j)}$ se conoce el valor correspondiente de cada variable x_k , ya que los datos originales contienen el vector completo $\mathbf{x}$. Con estos pares (θ, x_k) se entrena un modelo de regresión simbólica para cada coordenada $k = 2, \dots, n$. La función obtenida para la coordenada k de la solución j se denota como $g_k^{(j)}(\theta)$.

Ejemplo 7. Retomando el Ejemplo 4, supóngase que la búsqueda iterativa sobre x_1 ha producido dos expresiones: $g_1^{(1)}(a) = a$ con $\mathcal{S}^{(1)} = \{2, 4, \dots\}$, y $g_1^{(2)}(a) = a + 1$ con $\mathcal{S}^{(2)} = \{2, 5, \dots\}$. Para completar la primera solución, se toman los puntos de $\mathcal{S}^{(1)}$:

a	x_2
2	4
4	16
$\vdots$	$\vdots$

Con estos pares (a, x_2) se entrena regresión simbólica, que podría descubrir $g_2^{(1)}(a) = a^2$. Para la segunda solución, los puntos de $\mathcal{S}^{(2)}$ dan pares como $(a = 2, x_2 = 9)$ y $(a = 5, x_2 = 36)$, de donde se obtendría $g_2^{(2)}(a) = (a + 1)^2$.

Este procedimiento es posible porque, por construcción, dentro de un mismo subconjunto $\mathcal{S}^{(j)}$ cada θ aparece una sola vez: todos los puntos que están en $\mathcal{S}^{(j)}$ han sido explicados por la misma expresión $g_1^{(j)}$, y por tanto corresponden a la misma solución del sistema. En consecuencia, no existe la ambigüedad de múltiples valores de x_k para un mismo θ dentro de $\mathcal{S}^{(j)}$, y la regresión simbólica puede aplicarse sin necesidad de estrategias iterativas adicionales. Si excepcionalmente dos soluciones distintas compartieran el mismo valor de x_1 para algún θ , esta inconsistencia sería detectada en la fase de validación (Sección 2.3.4).

Una vez completado este proceso para todos los subconjuntos, se dispone de un conjunto de funciones que describen cada solución del sistema. Para la solución j , las

expresiones son

$$\begin{aligned}x_1 &= g_1^{(j)}(\boldsymbol{\theta}), \\x_2 &= g_2^{(j)}(\boldsymbol{\theta}), \\&\vdots \\x_n &= g_n^{(j)}(\boldsymbol{\theta}).\end{aligned}$$

Estas funciones constituyen un candidato a descripción analítica de la solución, que debe ser validado antes de ser aceptado definitivamente, como se explica en la sección siguiente.

2.3.4. Validación de una solución completa

Una vez obtenidas las expresiones simbólicas para todas las coordenadas de una candidata a solución, es necesario verificar que el conjunto de funciones

$$\mathbf{g}^{(j)}(\boldsymbol{\theta}) = (g_1^{(j)}(\boldsymbol{\theta}), g_2^{(j)}(\boldsymbol{\theta}), \dots, g_n^{(j)}(\boldsymbol{\theta}))$$

realmente satisface el sistema original $\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}$ sobre el subconjunto de parámetros $\mathcal{S}^{(j)}$ que la generó. Esta comprobación es imprescindible, ya que las expresiones se han obtenido por ajuste independiente de cada coordenada y podrían no ser compatibles entre sí al ser evaluadas conjuntamente en el sistema.

Para realizar la validación se sustituyen las funciones simbólicas en el sistema y se evalúa el valor

$$\mathbf{F}(\mathbf{g}^{(j)}(\boldsymbol{\theta}); \boldsymbol{\theta})$$

para cada tupla $\boldsymbol{\theta} \in \mathcal{S}^{(j)}$. A continuación se calcula el promedio de las normas euclidianas de estos valores sobre todo el subconjunto:

$$E^{(j)} = \frac{1}{|\mathcal{S}^{(j)}|} \sum_{\boldsymbol{\theta} \in \mathcal{S}^{(j)}} \|\mathbf{F}(\mathbf{g}^{(j)}(\boldsymbol{\theta}); \boldsymbol{\theta})\|_2.$$

Si el error medio $E^{(j)}$ es menor que una tolerancia predefinida ε_{val} , la solución se considera válida y se incorpora a la lista de resultados definitivos. Los puntos del subconjunto $\mathcal{S}^{(j)}$ se retiran del conjunto de datos, ya que han sido explicados por una expresión analítica aceptada.

Ejemplo 8. En el Ejemplo 7, la primera solución candidata es $x_1 = a$, $x_2 = a^2$, obtenida a partir de $\mathcal{S}^{(1)}$. Para validarla, se evalúa el sistema original $x_1^2 - x_2 = 0$ y $x_2 - ax_1 = 0$ en los puntos de $\mathcal{S}^{(1)}$:

a	$x_1 = a$	$x_2 = a^2$	$E^{(1)}$
2	2	4	0
4	4	16	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Al sustituir las expresiones se obtiene $a^2 - a^2 = 0$ y $a^2 - a \cdot a = 0$, por lo que $E^{(1)} \approx 0$ y la solución se acepta.

Si, por el contrario, el error medio supera la tolerancia, la solución se descarta y los puntos del subconjunto $\mathcal{S}^{(j)}$ vuelven al conjunto de datos. De esta forma, el algoritmo iterativo descrito en el paso 5 del pseudocódigo podrá considerarlos de nuevo en iteraciones posteriores, dando la oportunidad de descubrir otras expresiones que sí los expliquen satisfactoriamente.

De esta manera, el proceso de validación actúa como un filtro que garantiza que solo se conservan las descripciones que reproducen fielmente el comportamiento del sistema original. En el siguiente apartado se presentan las funciones de pérdida con las que se guía la búsqueda de las expresiones que luego se validan.

2.3.5. Funciones de pérdida utilizadas

La regresión simbólica emplea una función de pérdida que mide la discrepancia entre las predicciones y los valores observados. En este trabajo se evaluaron tres alternativas.

- **Error cuadrático medio (MSE).** Definido en la sección correspondiente, penaliza con mayor peso las desviaciones grandes.
- **Conteo de coincidencias.** Función discreta que asigna el mismo valor a todas las predicciones cuya diferencia con el dato observado esté por debajo de una tolerancia ε , y un valor constante mayor en caso contrario.
- **Sigmoidal.** Variante suave de la pérdida por conteo que transforma la diferencia absoluta mediante una sigmoide parametrizada para conservar la noción de umbral y, al mismo tiempo, ofrecer un gradiente útil para la búsqueda evolutiva.

La comparación experimental de estas tres variantes y la elección de la más adecuada se presentan en el Capítulo 3, junto con el resto de los experimentos realizados. En la sección siguiente se recoge el pseudocódigo completo del algoritmo, que integra todas las fases descritas hasta aquí. Los detalles de la implementación en Python, incluyendo la descripción de los módulos que componen el sistema y el formato de entrada y salida, se recogen en el Anexo.

2.4. Pseudocódigo del proceso principal

El procedimiento descrito en las secciones anteriores se integra en un algoritmo que toma como punto de partida la definición del sistema de ecuaciones y devuelve las expresiones analíticas para cada variable en cada una de las soluciones encontradas. A continuación se presenta el pseudocódigo global.

Pseudocódigo 1:

- Entrada:** sistema $\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}$,
variables $\mathbf{x}$,
parámetros $\boldsymbol{\theta}$,
intervalos para parámetros y para puntos iniciales,
tolerancias,
hiperparámetros de PySR.
- Salida:** lista de soluciones, cada una con una expresión simbólica por variable.
1. Definir cuadrícula de parámetros $\mathcal{G}$ mediante producto cartesiano.
 2. Para cada $\boldsymbol{\theta} \in \mathcal{G}$:
 Resolver numéricamente con múltiples puntos iniciales.
 Filtrar soluciones duplicadas usando distancia euclidiana.
 3. Combinar todos los pares $(\boldsymbol{\theta}, \mathbf{x})$ en un solo conjunto de datos.
 4. Tomar x_1 como guía para separar las soluciones.
 5. Aplicar búsqueda iterativa sobre x_1 para obtener una lista de expresiones $g_1^{(1)}, g_1^{(2)}, \dots$ y los subconjuntos de parámetros que cada una cubre.
 6. Para cada expresión $g_1^{(j)}$ y su subconjunto de parámetros $\mathcal{S}^{(j)}$:
 Para cada coordenada restante x_k con $k \neq 1$:
 Entrenar regresión simbólica sobre $\mathcal{S}^{(j)}$.
 Construir la función completa $\mathbf{g}^{(j)}(\boldsymbol{\theta})$.
 Validar calculando el error medio.
 Si la validación es satisfactoria, añadir la solución a la salida y retirar los puntos de $\mathcal{S}^{(j)}$ del conjunto de datos.
 7. Repetir los pasos 5–6 hasta que el número de puntos restantes sea menor que un umbral mínimo.

Los hiperparámetros mencionados (tolerancias, número de intentos, parámetros de

PySR) se detallan en el próximo capítulo, donde se justifican los valores seleccionados.

Capítulo 3

Experimentación y resultados

En este capítulo se presentan los experimentos realizados para validar la metodología propuesta en el Capítulo 2. Se persigue determinar en qué medida el algoritmo iterativo basado en regresión simbólica es capaz de descubrir, a partir de ceros numéricos encontrados, las distintas funciones que describen la dependencia de las soluciones respecto de los parámetros.

La evaluación se organiza en dos partes. En la primera se aborda un estudio comparativo de las funciones de pérdida —error cuadrático medio, pérdida por conteo de coincidencias y pérdida sigmoideal—, utilizando un conjunto de ecuaciones con una sola variable y con múltiples soluciones. El propósito de esta parte es identificar cuál de las funciones de pérdida resulta más adecuada para el problema de separación de raíces. En la segunda parte se selecciona la función de pérdida con mejor desempeño y se aplica la metodología completa a un conjunto de sistemas de ecuaciones no lineales paramétricos con varias variables y parámetros, que representan el escenario para el que fue concebido el algoritmo.

Los resultados obtenidos muestran que la pérdida sigmoideal supera a las otras dos alternativas tanto en la tasa de éxito como en la cobertura de puntos, y que el algoritmo es capaz de recuperar con alta fidelidad las expresiones analíticas de sistemas de diversa complejidad, incluso cuando las distintas soluciones se entremezclan en el conjunto de entrenamiento. En las secciones que siguen se describe el diseño de los experimentos, se detallan los problemas de prueba utilizados, se analizan los resultados desde distintas perspectivas y se discuten las limitaciones observadas.

3.1. Diseño experimental

Todos los experimentos que se presentan en este capítulo comparten una misma configuración base del algoritmo de regresión simbólica, descrita en el Capítulo 2. El motor de regresión simbólica utilizado es PySR [15].

Los hiperparámetros del algoritmo evolutivo se fijaron en los valores que se indican en la Tabla 3.1. Estos valores fueron seleccionados tras experimentos preliminares sobre ecuaciones con una sola variable y se mantuvieron constantes en todas las pruebas, salvo la función de pérdida, que varía en la primera parte para comparar su efecto y se fija en la segunda parte a la que resultó más efectiva.

Tabla 3.1: Hiperparámetros comunes utilizados en todos los experimentos.

Parámetro	Símbolo	Valor
Tamaño de la población	P	50
Número de iteraciones (generaciones)	G	500
Número de poblaciones independientes		30
Ciclos evolutivos por iteración		550
Tamaño máximo del árbol	$S_{\text{máx}}$	25
Coefficiente de parsimonia	α	0.0
Operadores unarios		neg, safe_sqrt, safe_cbrt, safe_root4–safe_root10
Operadores binarios		+, -, *, /, safe_pow

Los operadores unarios `neg`, `safe_sqrt`, `safe_cbrt` y `safe_root4` a `safe_root10` proporcionan, respectivamente, el cambio de signo, la raíz cuadrada, la raíz cúbica y las raíces cuarta a décima. Todas estas raíces se implementan de forma protegida: se aplican sobre el valor absoluto del argumento, y en los casos de raíz cúbica y de índice impar se preserva el signo original del operando, evitando así valores indefinidos durante la búsqueda evolutiva.

Para cada sistema o ecuación de prueba se construyó una cuadrícula de parámetros con 50 puntos equiespaciados por dimensión, que se combinaron mediante producto cartesiano. La resolución numérica de cada tupla se realizó con `SciPy` [20] a partir de 20 puntos iniciales aleatorios. Las soluciones duplicadas se filtraron usando la distancia euclidiana con una tolerancia $\tau = 10^{-3}$, y solo se conservaron aquellas cuyo residuo $\|\mathbf{F}(\mathbf{x}; \boldsymbol{\theta})\|_2$ fuera menor que 10^{-6} .

Los resultados de cada experimento se evaluaron mediante las siguientes métricas:

- **Éxito del caso:** se considera que un sistema (o ecuación) ha sido resuelto con éxito si el algoritmo encuentra exactamente el número de soluciones esperado y todas ellas superan el paso de validación descrito en la Sección 2.3.4. Se reporta la proporción de casos exitosos sobre el total.
- **Proporción de soluciones recuperadas:** para cada sistema se calcula el cociente entre el número de soluciones correctamente identificadas y el número

total de soluciones. Los valores se promedian sobre todos los sistemas para obtener una medida global de la capacidad del algoritmo para descubrir el conjunto completo de soluciones.

- **Tiempo de ejecución:** tiempo total en segundos empleado por el algoritmo desde la generación de la cuadrícula de parámetros hasta la obtención de las expresiones simbólicas finales.

En las secciones siguientes se aplican estas métricas a las dos partes de la experimentación: la comparación de funciones de pérdida sobre ecuaciones con una sola variable y la evaluación del algoritmo completo sobre sistemas de ecuaciones.

3.2. Comparación de funciones de pérdida con ecuaciones individuales

El objetivo de esta primera parte es evaluar el comportamiento de tres funciones de pérdida distintas dentro del mismo algoritmo de regresión simbólica, aplicado a ecuaciones paramétricas. Los resultados permitirán seleccionar la función de pérdida más adecuada para la parte siguiente, donde se trabaja con sistemas de ecuaciones.

3.2.1. Ecuaciones de prueba

Se utilizaron 30 ecuaciones de una sola variable y uno o dos parámetros. Las ecuaciones involucran expresiones lineales, cuadráticas, cúbicas y racionales, con el fin de cubrir una variedad representativa de comportamientos y niveles de complejidad. La Tabla 3.2 recoge las ecuaciones empleadas junto con las expresiones de sus soluciones.

Tabla 3.2: Ecuaciones utilizadas en la parte 1.

ID	Ecuación	Soluciones
L01	$x + a - 2 = 0$	$x = 2 - a$
L02	$2x - a = 0$	$x = a/2$
L03	$x - 3a + 5 = 0$	$x = 3a - 5$
L04	$3x + a - 6 = 0$	$x = (6 - a)/3$
L05	$x - (a + 1) = 0$	$x = a + 1$
L06	$x - (a + b) = 0$	$x = a + b$
L07	$x - 2a + 3b = 0$	$x = 2a - 3b$
L08	$x - ab = 0$	$x = ab$
L09	$ax - b = 0$	$x = b/a$

ID	Ecuación	Soluciones
L10	$x - a - b - c = 0$	$x = a + b + c$
Q11	$x^2 - a = 0$	$x = \sqrt{a}, x = -\sqrt{a}$
Q12	$x^2 - 2ax + a^2 = 0$	$x = a$
Q13	$(x - a)(x + a) = 0$	$x = a, x = -a$
Q14	$x^2 + ax = 0$	$x = 0, x = -a$
Q15	$x^2 - 3ax + 2a^2 = 0$	$x = a, x = 2a$
Q16	$x^2 - 2x - a = 0$	$x = 1 \pm \sqrt{1 + a}$
Q17	$ax^2 - 1 = 0$	$x = \pm 1/\sqrt{a}$
Q18	$x^2 - ax - b = 0$	$x = \frac{a \pm \sqrt{a^2 + 4b}}{2}$
Q19	$ax^2 + bx + c = 0$	$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
Q20	$(x - a)(x - b) = 0$	$x = a, x = b$
Q21	$x^2 - (a + b)x + ab = 0$	$x = a, x = b$
Q22	$x^2 - a^2 - b^2 = 0$	$x = \pm \sqrt{a^2 + b^2}$
Q23	$x^2 - 4ab = 0$	$x = \pm 2\sqrt{ab}$
Q24	$x^2 + ax + a^2/4 = 0$	$x = -a/2$
C25	$x^3 - a = 0$	$x = \sqrt[3]{a}$
C26	$x(x - a)(x + a) = 0$	$x = 0, x = \pm a$
C27	$(x - a)(x^2 + 1) = 0$	$x = a$
S36	$ax - 1 = 0$	$x = 1/a$
S37	$ax - b = 0$	$x = b/a$
S40	$(a + b)x - c = 0$	$x = c/(a + b)$

Para cada ecuación se generó una cuadrícula de parámetros con 50 puntos equiespaciados en el intervalo $[0.1, 5]$ para cada parámetro. Cuando la ecuación involucraba dos parámetros o más, se utilizó el producto cartesiano de los intervalos. Las soluciones se obtuvieron mediante el procedimiento descrito en la Sección 3.1.

3.2.2. Funciones de pérdida evaluadas

Se ejecutó el algoritmo iterativo de regresión simbólica (Sección 2.3) en tres configuraciones que solo diferían en la función de pérdida utilizada:

- **Error cuadrático medio (MSE):** función clásica que penaliza el cuadrado de la diferencia entre el valor estimado por la expresión y el valor observado:

$$\text{MSE}(f) = \frac{1}{N} \sum_{i=1}^N (v_i - f(\mathbf{u}_i))^2.$$

- **Cantidad de coincidencias:** función que contabiliza directamente cuántos puntos del conjunto de entrenamiento son estimados por la expresión dentro de una tolerancia $\varepsilon = 10^{-4}$:

$$\mathcal{L}_{\text{match}}(f) = 1 - \frac{1}{N} \sum_{i=1}^N \mathbb{I}(|v_i - f(\mathbf{u}_i)| < \varepsilon).$$

- **Sigmoidal:** función de pérdida basada en una transformación sigmoide con parámetros $\varepsilon = 0.005$ y $k = 100$, que suaviza la transición de la pérdida por conteo y proporciona una señal más informativa que la del conteo de coincidencias durante la búsqueda evolutiva:

$$\mathcal{L}_{\text{sig}}(f) = 1 - \frac{1}{N} \sum_{i=1}^N \sigma(k(\varepsilon - |v_i - f(\mathbf{u}_i)|)), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

A continuación se presentan los resultados obtenidos con cada una de estas configuraciones.

3.2.3. Resultados

Los resultados globales de las tres configuraciones se resumen en la Tabla 3.3. Se reporta el número y la proporción de casos exitosos, la proporción media de soluciones recuperadas, el tiempo total de ejecución acumulado y el tiempo promedio por ecuación sobre las 30 ecuaciones.

Tabla 3.3: Comparación de funciones de pérdida sobre las 30 ecuaciones.

Métrica	MSE	Match count	Sigmoidal
Casos exitosos	18 (60%)	24 (80%)	27 (90%)
Prop. soluciones recuperadas (media)	0.61	0.83	0.92
Tiempo total (s)	5 987	14 718	5 322
Tiempo promedio por ecuación (s)	199.6	490.6	177.4

La función sigmoidal consiguió la tasa más alta de casos completamente resueltos y la mayor fracción de soluciones recuperadas. Además, fue la más rápida de las tres, con un tiempo total inferior al del MSE y sustancialmente menor que el del conteo de coincidencias.

A continuación se analizan con más detalle estos resultados y se discuten las razones que explican el mejor desempeño de la pérdida sigmoidal.

3.2.4. Análisis

Los datos de la Tabla 3.3 muestran que la pérdida sigmoideal obtiene numéricamente la tasa de éxito más alta y la mayor proporción de soluciones recuperadas. Para determinar si estas diferencias son estadísticamente significativas, se realizaron pruebas t de Student pareadas de una cola comparando las tres funciones de pérdida sobre los 30 casos.

En cuanto al éxito del proceso, la sigmoideal supera al MSE de forma significativa ($t = 3.53$, $p = 0.0007$), lo que permite rechazar la hipótesis nula de que el desempeño de la sigmoideal no es superior. En cambio, la comparación entre la sigmoideal y el conteo de coincidencias ($t = 1.44$, $p = 0.08$) no alcanza el nivel de significancia $\alpha = 0.05$, por lo que la evidencia estadística no es suficiente para afirmar que la sigmoideal supere al conteo de coincidencias en tasa de éxito, aunque la diferencia numérica sea apreciable (90 % frente a 80 %).

Respecto al tiempo de ejecución, la sigmoideal es significativamente más rápida que el conteo de coincidencias ($t = -4.79$, $p < 0.001$), pero no se detecta una diferencia significativa con el MSE ($t = -0.69$, $p = 0.25$). El tiempo elevado del conteo de coincidencias se debe en gran parte a dos ecuaciones (L03 y L04) donde el algoritmo agotó el tiempo límite sin converger, lo que penaliza fuertemente el promedio.

Desde el punto de vista cualitativo, la ventaja de la pérdida sigmoideal sobre el MSE se explica porque este último tiende a promediar las distintas soluciones cuando los datos no están separados, produciendo expresiones que no corresponden a ninguna de ellas. La pérdida por conteo de coincidencias no sufre este problema, pero su naturaleza binaria puede provocar estancamientos durante la búsqueda evolutiva. La sigmoide combina ambas ventajas: preserva la noción de umbral propia del conteo de coincidencias, pero ofrece una señal gradual que guía mejor la exploración del espacio de expresiones.

Es importante señalar que los resultados aquí presentados corresponden a tres ejecuciones del algoritmo por cada caso y función de pérdida. Dado que la regresión simbólica basada en programación genética es un proceso estocástico, un número limitado de corridas puede no capturar completamente la variabilidad del desempeño esperado. En particular, los tres casos donde la sigmoideal no encontró solución no la encontraron en ninguna de las tres ejecuciones (10 % del total), lo que sugiere que la dificultad es intrínseca a esas ecuaciones y no producto de la aleatoriedad del algoritmo.

A la vista de estos resultados, se selecciona la pérdida sigmoideal como la función de pérdida a utilizar en el resto de los experimentos. En la sección siguiente se aplica la metodología completa a sistemas de ecuaciones no lineales paramétricos, que constituyen el escenario para el que fue concebido el algoritmo.

3.3. Extensión a sistemas de ecuaciones

Una vez seleccionada la función de pérdida sigmoideal como la más efectiva en ecuaciones con una sola variable, se procede a evaluar la metodología completa sobre sistemas de ecuaciones no lineales paramétricos, que constituyen el escenario para el que fue concebido el algoritmo. En esta parte se trabaja con sistemas de dos variables y hasta tres parámetros, que presentan múltiples soluciones para una misma tupla de parámetros.

A continuación se describen los sistemas de prueba utilizados (Sección 3.3.1), la configuración específica del algoritmo para esta fase (Sección ??) y los resultados obtenidos (Sección 3.3.3), que luego se analizan en detalle (Sección 3.3.4).

3.3.1. Sistemas de prueba

Se emplearon 30 sistemas de ecuaciones no lineales paramétricos que cubren estructuras algebraicas variadas y distinto número de soluciones. Todos los sistemas tienen dos variables, que se denotan x e y , y entre uno y tres parámetros, denotados por letras iniciales del alfabeto (a, b, c) . La Tabla 3.4 recoge las ecuaciones de cada sistema junto con el número de soluciones reales que admite para cada valor de los parámetros.

Tabla 3.4: Sistemas de ecuaciones utilizados en la parte 2.

ID	Sistema	Soluciones
S01	$\begin{cases} x + y - a = 0 \\ x - y - b = 0 \end{cases}$	$\begin{aligned} x &= (a + b)/2 \\ y &= (a - b)/2 \end{aligned}$
S02	$\begin{cases} 2x - y - 3a + 3b = 0 \\ x + y - 3a = 0 \end{cases}$	$\begin{aligned} x &= 2a - b \\ y &= a + b \end{aligned}$
S03	$\begin{cases} ax + by - c = 0 \\ ax - by + c = 0 \end{cases}$	$\begin{aligned} x &= 0 \\ y &= c/b \end{aligned}$
S04	$\begin{cases} 2x - y - a - 5 = 0 \\ x + y - 8a - 1 = 0 \end{cases}$	$\begin{aligned} x &= 3a + 2 \\ y &= 5a - 1 \end{aligned}$
S05	$\begin{cases} (b + c)x + (a - c)y - a - b = 0 \\ (b + c)x - (a - c)y - a + b = 0 \end{cases}$	$\begin{aligned} x &= a/(b + c) \\ y &= b/(a - c) \end{aligned}$
S06	$\begin{cases} x^2 - a = 0 \\ y - x = 0 \end{cases}$	$\begin{aligned} 1: x &= \sqrt{a}, y = \sqrt{a} \\ 2: x &= -\sqrt{a}, y = -\sqrt{a} \end{aligned}$
S07	$\begin{cases} x^2 - (a + b)x + ab = 0 \\ y - 2x = 0 \end{cases}$	$\begin{aligned} 1: x &= a, y = 2a \\ 2: x &= b, y = 2b \end{aligned}$

ID	Sistema	Soluciones
S08	$\begin{cases} (x^2 - a^2)(x^2 - b^2) = 0 \\ y - x^2 - x = 0 \end{cases}$	1: $x = a, y = a^2 + a$ 2: $x = -a, y = a^2 - a$ 3: $x = b, y = b^2 + b$ 4: $x = -b, y = b^2 - b$
S09	$\begin{cases} x^2 - y^2 - a = 0 \\ x - 2y - b = 0 \end{cases}$	1: $x = \frac{-b+2\sqrt{3a+b^2}}{3}, y = \frac{-2b+\sqrt{3a+b^2}}{3}$ 2: $x = \frac{-b-2\sqrt{3a+b^2}}{3}, y = \frac{-2b-\sqrt{3a+b^2}}{3}$
S10	$\begin{cases} x^2 - ax = 0 \\ y - 2x + b = 0 \end{cases}$	1: $x = 0, y = -b$ 2: $x = a, y = 2a - b$
S11	$\begin{cases} x^4 - 5ax^2 + 4a^2 = 0 \\ y - x^2 - x - b = 0 \end{cases}$	1: $x = \sqrt{a}, y = a + \sqrt{a} + b$ 2: $x = -\sqrt{a}, y = a - \sqrt{a} + b$ 3: $x = 2\sqrt{a}, y = 4a + 2\sqrt{a} + b$ 4: $x = -2\sqrt{a}, y = 4a - 2\sqrt{a} + b$
S12	$\begin{cases} (x-a)(x-b)(x+a+b) = 0 \\ y - 2x + c = 0 \end{cases}$	1: $x = a, y = 2a - c$ 2: $x = b, y = 2b - c$ 3: $x = -a - b, y = -2a - 2b - c$
S13	$\begin{cases} x^3 - a = 0 \\ y - x^2 = 0 \end{cases}$	$x = \sqrt[3]{a}$ $y = a^{2/3}$
S14	$\begin{cases} 2x + y - 3 = 0 \\ xy - a = 0 \end{cases}$	1: $x = \frac{3+\sqrt{9-8a}}{4}, y = \frac{3-\sqrt{9-8a}}{2}$ 2: $x = \frac{3-\sqrt{9-8a}}{4}, y = \frac{3+\sqrt{9-8a}}{2}$
S15	$\begin{cases} (x-a)(x-b)(x-a-b) = 0 \\ x + y - a = 0 \end{cases}$	1: $x = a, y = 0$ 2: $x = b, y = a - b$ 3: $x = a + b, y = -b$
S16	$\begin{cases} xy - 1 = 0 \\ xb - ya = 0 \end{cases}$	1: $x = \sqrt{a/b}, y = \sqrt{b/a}$ 2: $x = -\sqrt{a/b}, y = -\sqrt{b/a}$
S17	$\begin{cases} x^2 - y^2 - 4ab = 0 \\ xy - a^2 + b^2 = 0 \end{cases}$	1: $x = a + b, y = a - b$ 2: $x = -a - b, y = -a + b$
S18	$\begin{cases} (b+c)x + (a+b)y - a - c = 0 \\ (b+c)x - (a+b)y - a + c = 0 \end{cases}$	$x = a/(b+c)$ $y = c/(a+b)$
S19	$\begin{cases} (x^2 - a^2)(x^2 - (a+b)^2) = 0 \\ y - x^2 - 2x - c = 0 \end{cases}$	1: $x = a, y = a^2 + 2a + c$ 2: $x = -a, y = a^2 - 2a + c$ 3: $x = a + b, y = (a+b)^2 + 2(a+b) + c$ 4: $x = -a - b, y = (a+b)^2 - 2(a+b) + c$
S20	$\begin{cases} x^2 - 2x - a = 0 \\ y^2 - x = 0 \end{cases}$	1: $x = 1 + \sqrt{1+a}, y = \sqrt{1 + \sqrt{1+a}}$ 2: $x = 1 - \sqrt{1+a}, y = \sqrt{1 - \sqrt{1+a}}$

ID	Sistema	Soluciones
S21	$\begin{cases} xy - a = 0 \\ ay - bx = 0 \end{cases}$	1: $x = \sqrt{a^2/b}, y = \sqrt{b}$ 2: $x = -\sqrt{a^2/b}, y = -\sqrt{b}$
S22	$\begin{cases} 3x + y - a = 0 \\ xy - b = 0 \end{cases}$	1: $x = \frac{a + \sqrt{a^2 - 12b}}{6}, y = \frac{a - \sqrt{a^2 - 12b}}{2}$ 2: $x = \frac{a - \sqrt{a^2 - 12b}}{6}, y = \frac{a + \sqrt{a^2 - 12b}}{2}$
S23	$\begin{cases} (x^2 - a)(x^2 - 2a) = 0 \\ y - x^3 - 2x = 0 \end{cases}$	1: $x = \sqrt{a}, y = a\sqrt{a} + 2\sqrt{a}$ 2: $x = -\sqrt{a}, y = -a\sqrt{a} - 2\sqrt{a}$ 3: $x = \sqrt{2a}, y = 2a\sqrt{2a} + 2\sqrt{2a}$ 4: $x = -\sqrt{2a}, y = -2a\sqrt{2a} - 2\sqrt{2a}$
S24	$\begin{cases} (x - a)(x - b) = 0 \\ y - x^2 + c = 0 \end{cases}$	1: $x = a, y = a^2 - c$ 2: $x = b, y = b^2 - c$
S25	$\begin{cases} (x^2 - (a/b)^2)(x^2 - (b/a)^2) = 0 \\ y - x^2 + 3x - a = 0 \end{cases}$	1: $x = a/b, y = (a/b)^2 - 3(a/b) + a$ 2: $x = -a/b, y = (a/b)^2 + 3(a/b) + a$ 3: $x = b/a, y = (b/a)^2 - 3(b/a) + a$ 4: $x = -b/a, y = (b/a)^2 + 3(b/a) + a$
S26	$\begin{cases} x^2 + y^2 - 1 = 0 \\ x \cos a - y \sin a = 0 \end{cases}$	1: $x = \sin a, y = \cos a$ 2: $x = -\sin a, y = -\cos a$
S27	$\begin{cases} 2x + y - e^a = 0 \\ x - y - e^b = 0 \end{cases}$	$x = (e^a + e^b)/3$ $y = (e^a - 2e^b)/3$
S28	$\begin{cases} x^2 + y^2 - a^2 = 0 \\ x \sin b - y \cos b = 0 \end{cases}$	1: $x = a \cos b, y = a \sin b$ 2: $x = -a \cos b, y = -a \sin b$
S29	$\begin{cases} xy - 1 = 0 \\ x - ye^{2a} = 0 \end{cases}$	1: $x = e^a, y = e^{-a}$ 2: $x = -e^a, y = -e^{-a}$
S30	$\begin{cases} x^2 + y^2 - e^{2a} = 0 \\ x \sin b - y \cos b = 0 \end{cases}$	1: $x = e^a \cos b, y = e^a \sin b$ 2: $x = -e^a \cos b, y = -e^a \sin b$

Los sistemas S26–S30 incorporan funciones trigonométricas y exponenciales, lo que amplía el espectro de no linealidades respecto a los sistemas puramente polinomiales y permite evaluar el comportamiento del algoritmo ante expresiones trascendentes.

Para cada sistema se generó una cuadrícula de parámetros con 50 puntos equiespaciados por dimensión dentro del intervalo especificado en los casos de prueba, que se combinaron mediante producto cartesiano. La resolución numérica y el filtrado de duplicados se realizaron según lo descrito en la Sección 3.1.

3.3.2. Configuración específica

Se utilizó exclusivamente la pérdida sigmoideal con los mismos parámetros que en la parte 1 ($\varepsilon = 0.005$, $k = 100$). Los hiperparámetros del algoritmo evolutivo fueron

los de la Tabla 3.1. Para la separación por coordenadas, explicada en la Sección 2.3.1, se tomó siempre la primera variable como punto de partida.

3.3.3. Resultados

La Tabla 3.5 presenta los resultados obtenidos con la pérdida sigmoideal para cada uno de los 30 sistemas. Se indica el número de soluciones esperadas, el número de soluciones encontradas, el porcentaje de soluciones recuperadas y el tiempo de ejecución.

Tabla 3.5: Resultados de la parte 2 con pérdida sigmoideal.

ID	Sols. esperadas	Sols. encontradas	Recuperadas (%)	Tiempo (s)
S01	1	1	100	194.2
S02	1	1	100	203.0
S03	1	1	100	278.2
S04	1	1	100	190.2
S05	1	1	100	278.0
S06	2	2	100	472.7
S07	2	2	100	487.3
S08	4	4	100	447.8
S09	2	0	0	646.7
S10	2	2	100	188.5
S11	4	4	100	694.2
S12	3	3	100	311.3
S13	1	1	100	125.6
S14	2	0	0	512.3
S15	3	3	100	373.7
S16	2	2	100	307.7
S17	2	2	100	517.6
S18	1	1	100	328.6
S19	4	4	100	661.6
S20	2	2	100	356.8
S21	2	2	100	325.2
S22	2	0	0	329.2
S23	4	4	100	860.1
S24	2	2	100	1352.9
S25	4	4	100	609.1
S26	2	2	100	290.9
S27	1	1	100	139.9

ID	Sols. esperadas	Sols. encontradas	Recuperadas (%)	Tiempo (s)
S28	2	2	100	325.3
S29	2	2	100	276.5
S30	2	2	100	336.2

De los 30 sistemas, 27 fueron resueltos con todas las soluciones recuperadas (90 % de éxito). En los casos exitosos, las expresiones encontradas superaron la validación y cubrieron prácticamente todos los puntos del conjunto de entrenamiento. En los tres sistemas restantes (S09, S14 y S22) el algoritmo no consiguió encontrar ninguna expresión que cumpliera el criterio de validación.

Para complementar la evaluación, los mismos 30 sistemas se ejecutaron también con la función de pérdida por conteo de coincidencias, con idénticos hiperparámetros. La Tabla 3.6 resume la comparación.

Tabla 3.6: Comparación de funciones de pérdida sobre los 30 sistemas.

Métrica	Sigmoidal	Match count
Casos exitosos	27 (90 %)	21 (70 %)
Prop. soluciones recuperadas (media)	0.90	0.70
Tiempo total (s)	12 421	18 987

Se realizó una prueba t pareada de una cola para comparar la tasa de éxito de ambas funciones. La sigmoidal supera al conteo de coincidencias de forma estadísticamente significativa ($t = 1.99$, $p = 0.028$). En cuanto al tiempo de ejecución, también la sigmoidal resultó significativamente más rápida ($t = -2.50$, $p = 0.009$).

3.3.4. Análisis

La combinación de la metodología propuesta con la función de pérdida sigmoidal resuelve con éxito 27 de los 30 sistemas de prueba, lo que representa un 90 % de los casos. En todas las ocasiones en que el algoritmo converge, recupera la totalidad de las soluciones, incluidas aquellas de sistemas con tres y cuatro expresiones distintas para una misma tupla de parámetros.

Los tres sistemas que no pudieron ser resueltos (S09, S14 y S22) se caracterizan por tener soluciones con estructuras especialmente complejas. La Tabla 3.7 muestra las expresiones analíticas esperadas para cada uno de ellos.

Tabla 3.7: Soluciones esperadas de los sistemas no resueltos.

ID	Soluciones esperadas
S09	$x = \frac{-b \pm 2\sqrt{3a+b^2}}{3}, y = \frac{-2b \pm \sqrt{3a+b^2}}{3}$
S14	$x = \frac{3 \pm \sqrt{9-8a}}{4}, y = \frac{3 \mp \sqrt{9-8a}}{2}$
S22	$x = \frac{a \pm \sqrt{a^2-12b}}{6}, y = \frac{a \mp \sqrt{a^2-12b}}{2}$

En los tres casos, las soluciones esperadas presentan un grado de complejidad simbólica elevado: involucran raíces cuadradas de expresiones que combinan parámetros de forma no trivial, junto con fracciones y signos opuestos. Descubrir expresiones de esta naturaleza mediante regresión simbólica exige ensamblar simultáneamente varios operadores —raíces, divisiones, sumas con signos alternados—, lo que dificulta la convergencia del proceso evolutivo. A ello se suma el que la presencia de múltiples soluciones para una misma tupla de parámetros obliga al algoritmo a separar correctamente los puntos antes de poder ajustar cada expresión, y cuando las distintas soluciones tienen estructuras algebraicas tan diferentes, esta separación puede resultar especialmente costosa. Cabe señalar que las soluciones esperadas de estos tres sistemas requieren árboles sintácticos de entre 10 y 16 nodos, tamaños muy por debajo del límite $S_{\text{máx}} = 25$ fijado en los experimentos, lo que indica que la dificultad para hallarlas no es de naturaleza estructural sino que radica en la convergencia misma del proceso evolutivo.

Es importante señalar que los resultados presentados corresponden a tres ejecuciones del algoritmo por cada sistema y función de pérdida. Aunque este número de réplicas ofrece cierta protección frente a fluctuaciones aleatorias, la naturaleza estocástica de la programación genética hace deseable la exploración de configuraciones más potentes (mayor tamaño máximo del árbol, más generaciones, poblaciones más grandes) sobre los sistemas que no pudieron ser resueltos, lo que constituye una línea natural de trabajo futuro.

En cuanto a la comparación con el conteo de coincidencias, la ventaja de la sigmoideal se mantiene también en sistemas de ecuaciones (90 % vs. 70 %), y la diferencia es estadísticamente significativa ($t = 1.99$, $p = 0.028$). Este resultado refuerza la conclusión de la parte 1: la señal suave que proporciona la sigmoide facilita la exploración del espacio de expresiones y reduce los estancamientos durante el proceso evolutivo.

3.4. Consideraciones finales

Los experimentos realizados han permitido validar la metodología propuesta para la recuperación de expresiones analíticas de los ceros de sistemas de ecuaciones no lineales paramétricos.

La comparación de funciones de pérdida mostró que la pérdida sigmoideal es la más adecuada para este problema, ya que evita tanto el promedio de soluciones que produce el MSE como los estancamientos que genera la pérdida por conteo. Con esta función, el algoritmo iterativo de regresión simbólica alcanzó un 90 % de éxito en los treinta sistemas de prueba y, en todos los casos exitosos, recuperó la totalidad de las soluciones, incluso cuando coexistían tres o cuatro soluciones distintas para una misma tupla de parámetros.

La mayoría de los sistemas evaluados son de naturaleza polinomial. No obstante, los sistemas S26–S30 incorporan funciones trigonométricas y exponenciales, y en todos ellos el algoritmo encontró correctamente las soluciones. Este resultado sugiere que el método puede extenderse a sistemas con no linealidades trascendentes, aunque una exploración más sistemática de este tipo de funciones queda como línea de trabajo futura.

Capítulo 4

Conclusiones

El presente trabajo tuvo como objetivo diseñar e implementar un algoritmo que, a partir de la definición de un sistema de ecuaciones no lineales paramétricas, recupere automáticamente las expresiones analíticas de todas sus soluciones mediante regresión simbólica. A continuación se resumen los principales resultados y aportes alcanzados.

Se construyó un procedimiento automático para generar el conjunto de datos de entrada para la regresión simbólica, basado en el producto cartesiano de los intervalos de los parámetros y la resolución numérica con múltiples puntos iniciales. El filtrado de soluciones duplicadas, realizado mediante un criterio de distancia euclidiana, garantiza que todas las soluciones distintas queden registradas sin repeticiones.

Se desarrolló un algoritmo iterativo que emplea regresión simbólica para descubrir, de manera iterativa, las expresiones analíticas que componen la solución de un sistema de ecuaciones paramétrico. Partiendo del conjunto de datos sin separar, el algoritmo identifica secuencialmente cada una de las soluciones y valida cada expresión frente al sistema original, retirando los puntos ya explicados antes de continuar con los restantes.

Se evaluaron tres funciones de pérdida —MSE, conteo de coincidencias y sigmoideal— sobre un conjunto de 30 ecuaciones univariadas con múltiples soluciones. Los resultados mostraron que la pérdida sigmoideal alcanza un 90% de éxito, superando al MSE y al conteo de coincidencias, tanto en tasa de casos resueltos como en cobertura de puntos.

La metodología completa se aplicó a 30 sistemas de ecuaciones no lineales paramétricos con dos variables y hasta tres parámetros. El algoritmo resolvió con éxito 27 de los 30 sistemas (90%), recuperando la totalidad de las soluciones en todos los casos exitosos, incluidos sistemas con tres y cuatro expresiones distintas para una misma tupla de parámetros.

Los tres sistemas que no pudieron ser resueltos presentan soluciones con raíces anidadas o combinaciones de cocientes. Las expresiones analíticas correspondientes

requieren árboles sintácticos cuyo tamaño está dentro del límite fijado en los experimentos, lo que indica que la dificultad para hallarlas no es de naturaleza estructural sino que radica en la convergencia misma del proceso evolutivo. Esto sugiere que el algoritmo es robusto para una amplia clase de sistemas, pero que ciertas configuraciones de parámetros evolutivos pueden dificultar el descubrimiento de expresiones con estructuras algebraicas particularmente complejas.

La arquitectura modular de la implementación facilita la extensibilidad del sistema y su aplicación a nuevos problemas sin modificar los módulos existentes.

En conjunto, los resultados experimentales validan la metodología propuesta y demuestran que la combinación de un algoritmo iterativo con la función de pérdida sigmoideal permite recuperar las expresiones analíticas que describen los ceros de sistemas de ecuaciones no lineales paramétricos.

Recomendaciones

A partir de los resultados obtenidos y de las limitaciones observadas, se proponen las siguientes líneas de trabajo futuro:

- Relajar las restricciones computacionales —aumentando el tamaño máximo del árbol, el número de iteraciones y la diversidad de la población— para abordar sistemas con soluciones de mayor complejidad.
- Explorar el comportamiento del algoritmo en sistemas con funciones trascendentes (trigonométricas, exponenciales) de forma más sistemática, ampliando los resultados obtenidos en los sistemas S26–S30.
- Explorar el uso de otras variantes de regresión simbólica y comparar su desempeño con el motor PySR empleado en este trabajo.
- Integrar el sistema desarrollado en una interfaz gráfica o en un flujo de trabajo automatizado que facilite su uso por parte de investigadores sin experiencia en programación.

Apéndice A

Anexos

A.1. Detalles de implementación

El algoritmo descrito en el Capítulo 2 se materializa en un sistema compuesto por varios módulos independientes, cada uno responsable de una etapa del proceso. A continuación se describen estos módulos, se detallan los recursos que requieren y se resume el formato de los resultados que producen.

A.2. Descripción de los módulos

La implementación sigue el orden del pipeline presentado en el pseudocódigo de la Sección 2.4. Los módulos principales son:

- `config.py`: centraliza todos los parámetros globales del experimento, como los intervalos de los parámetros, el número de puntos de la cuadrícula, las opciones del solucionador numérico (tolerancias, número de intentos) y los hiperparámetros del modelo de regresión simbólica (tamaño de la población, número de generaciones, coeficiente de parsimonia, etc.).
- `equation_parser.py`: convierte las ecuaciones del sistema, escritas como cadenas de texto, en expresiones simbólicas de `SymPy` [32] que pueden ser evaluadas numéricamente. Para sistemas de varias ecuaciones, este módulo procesa la lista completa y devuelve tanto las expresiones como un diccionario con todos los símbolos involucrados.
- `grid_generator.py`: construye el producto cartesiano de los conjuntos de valores de los parámetros (Sección 2.2), generando la cuadrícula $\mathcal{G}$ que se utiliza como entrada en la etapa de resolución numérica.

- `solver.py`: para cada tupla de la cuadrícula, resuelve numéricamente el sistema con SciPy [20] empleando múltiples puntos iniciales aleatorios. Aplica el filtro de duplicados basado en la distancia euclidiana (Sección 1.1) y devuelve la lista de soluciones únicas encontradas para cada valor de los parámetros.
- `root_grouping.py`: reorganiza los datos generados por el solucionador para que puedan ser utilizados por los módulos de regresión simbólica. Produce tanto el conjunto combinado de pares $(\theta, \mathbf{x})$ como, opcionalmente, la separación por soluciones de acuerdo con la estrategia descrita en la Sección 2.3.1.
- `symbolic_regression.py`: contiene la lógica de entrenamiento de PySR [15] y el algoritmo iterativo que, partiendo del conjunto de datos completo, descubre una a una las expresiones para x_1 (Sección 2.3.2). Tras cada entrenamiento, se examinan todas las ecuaciones generadas por el modelo y se selecciona aquella que cubre la mayor cantidad de puntos, descartando el resto antes de la siguiente iteración.
- `regression_adapter.py`: implementa la estrategia multidimensional: extiende las expresiones obtenidas para x_1 a las coordenadas restantes (Sección 2.3.3) y valida cada solución completa calculando el error medio sobre el sistema original (Sección 2.3.4).
- `main.py`: integra los módulos anteriores y ejecuta el pipeline completo, desde la lectura de la configuración hasta la escritura de los resultados.

Además de los módulos desarrollados, el sistema depende de las bibliotecas externas SymPy para la manipulación simbólica, SciPy para la resolución numérica y PySR para la regresión simbólica.

A.3. Entradas, parámetros y salidas del sistema

Para cerrar la descripción del método, se resumen a continuación los datos que requiere el sistema y los resultados que produce.

Entradas del sistema:

- Lista de ecuaciones que definen el sistema $\mathbf{F}(\mathbf{x}; \theta) = \mathbf{0}$.
- Nombres de las variables $\mathbf{x}$ y de los parámetros θ .
- Intervalos $[\theta_i^{\min}, \theta_i^{\max}]$ para cada parámetro, y número de puntos de la cuadrícula.

Parámetros principales:

- Número de puntos iniciales aleatorios y tolerancias para la resolución numérica $(\varepsilon_{\text{res}}, \tau)$.
- Función de pérdida seleccionada y sus parámetros (ε, k) (Sección 2.3.5).
- Hiperparámetros de PySR: tamaño de población, número de generaciones, coeficiente de parsimonia, tamaño máximo de árbol, etc. (Sección 1.2).

Salidas del sistema:

- Lista de soluciones encontradas. Cada solución consiste en un conjunto de expresiones simbólicas $\{g_1(\boldsymbol{\theta}), \dots, g_n(\boldsymbol{\theta})\}$ que describen las variables en función de los parámetros.
- Para cada solución se indica el subconjunto de parámetros sobre el cual fue validada y el número de puntos que cubre.

Bibliografía

- [1] Neil Gershenfeld. *The Nature of Mathematical Modeling*. Cambridge University Press, 2012 (vid. pág. 1).
- [2] Steven H Strogatz. *Nonlinear Dynamics and Chaos*. CRC Press, 2018 (vid. pág. 1).
- [3] Paul Horowitz y Winfield Hill. *The Art of Electronics*. Cambridge University Press, 2015 (vid. pág. 1).
- [4] Keith J Laidler. *Chemical Kinetics*. Harper & Row, 1987 (vid. pág. 1).
- [5] Eugene L. Allgower y Kurt Georg. *Numerical Continuation Methods: An Introduction*. Springer, 1990 (vid. pág. 1).
- [6] Andrea Saltelli et al. *Global Sensitivity Analysis: The Primer*. John Wiley & Sons, 2008 (vid. pág. 1).
- [7] Alexander IJ Forrester y Andy J Keane. *Engineering Design via Surrogate Modelling*. John Wiley & Sons, 2008 (vid. pág. 1).
- [8] MS Eldred y BJ Bichon. «Formulations for surrogate-based optimization under uncertainty». En: *49th AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*. AIAA 2009-2282. 2009 (vid. pág. 1).
- [9] Richard L. Burden y J. Douglas Faires. *Numerical Analysis*. 9th. Brooks/Cole, 2010 (vid. págs. 1, 6, 16).
- [10] Jorge Nocedal y Stephen J. Wright. *Numerical Optimization*. 2nd. Springer, 2006 (vid. págs. 1, 6).
- [11] I Meleshko. *Nonlinear Systems: Analysis, Stability, and Control*. Springer, 1991 (vid. pág. 1).
- [12] T Ojika, S Watanabe y T Mitsui. «Deflation algorithm for the multiple roots of a system of nonlinear equations». En: *Journal of Mathematical Analysis and Applications* 96.2 (1983), págs. 463-479 (vid. pág. 1).

- [13] Adam Bartnik et al. «MDCM: A Multi-Dimensional Continuation Method for Solving Parametric Nonlinear Systems». En: *Journal of Computational Science* (2024). Describes a multidimensional bisection method to find all real roots of a nonlinear system within a bounded domain. (vid. pág. 1).
- [14] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA, USA: MIT Press, 1992 (vid. págs. 1, 4, 9-11).
- [15] Miles Cranmer. «PySR: Fast & Parallelized Symbolic Regression in Python/Julia». En: *Journal of Open Source Software* 8.89 (2023), pág. 5308. DOI: 10.21105/joss.05308 (vid. págs. 1, 9, 12, 27, 44).
- [16] M. A. Diniz et al. «SymbolFit: Automatic Parametric Modeling with Symbolic Regression». En: *Information Sciences* (2025). Framework that automates parametric modeling using symbolic regression to find a single function that fits the entire dataset. (vid. pág. 1).
- [17] R. Goldman et al. «leaf: A Flexible Framework for Symbolic Regression in Complex Systems». En: *R Journal* (2024). Unified R package for symbolic regression in nonlinear systems that require model combination. (vid. pág. 1).
- [18] O. Ukorigho y O. Owoyele. «A Competitive Learning Approach for Specialized Models: A Solution for Symbolic Regression of Specialized Physics Systems». En: *Proceedings of the 2023 ...* (2023). Proposes competitive learning for symbolic regression when data contain multiple functional regimes. (vid. pág. 1).
- [19] Cyril Merlet et al. «Mixing Neural Networks, Continuation and Symbolic Computation to Solve Parametric Systems of Nonlinear Equations». En: *Mathematics and Computers in Simulation* (2024). Uses neural networks trained on numerical solutions to predict initial guesses for Newton's method; does not produce explicit analytical expressions. (vid. pág. 2).
- [20] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant et al. «SciPy 1.0: fundamental algorithms for scientific computing in Python». En: *Nature Methods* 17.3 (2020), págs. 261-272. DOI: 10.1038/s41592-019-0686-2 (vid. págs. 4, 7, 28, 44).
- [21] Charles G. Broyden. «A class of methods for solving nonlinear simultaneous equations». En: *Mathematics of Computation* 19.92 (1965), págs. 577-593 (vid. pág. 6).
- [22] Michael Schmidt y Hod Lipson. «Distilling Free-Form Natural Laws from Experimental Data». En: *Science* 324.5923 (2009), págs. 81-85. DOI: 10.1126/science.1165893 (vid. pág. 9).

- [23] Marco Virgolin, Tanja Alderliesten y Peter A. N. Bosman. «Symbolic Regression is NP-hard». En: *arXiv preprint arXiv:2207.01018* (2022) (vid. pág. 10).
- [24] Daniel Kantor, Fernando J. Von Zuben y Fabricio Olivetti de Franca. «Simulated Annealing for Symbolic Regression». En: *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '21)*. 2021. DOI: 10.1145/3449639.3459345 (vid. pág. 10).
- [25] Léo François Dal Piccol Sotto y Vinícius Veloso de Melo. «A Probabilistic Linear Genetic Programming with Stochastic Context-Free Grammar for Solving Symbolic Regression Problems». En: *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '17)*. 2017. DOI: 10.1145/3071178.3071263 (vid. pág. 10).
- [26] Allan Costa et al. «OccamNet: A Fast Neural Model for Symbolic Regression at Scale». En: *arXiv preprint arXiv:2007.10799* (2021) (vid. pág. 10).
- [27] Yusong Deng et al. «Operator Feature Neural Network for Symbolic Regression». En: *arXiv preprint arXiv:2408.07719* (2024) (vid. pág. 10).
- [28] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. New York: Springer, 2009 (vid. pág. 11).
- [29] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. New York: Springer, 2006 (vid. pág. 11).
- [30] Darrell Whitley, Soraya Rana y Robert B. Heckendorn. «The Island Model Genetic Algorithm: On Separability, Population Size and Convergence». En: *Journal of Computing and Information Technology* (1990) (vid. pág. 12).
- [31] Douglas C. Montgomery y George C. Runger. *Applied Statistics and Probability for Engineers*. 7.^a ed. Wiley, 2017 (vid. pág. 13).
- [32] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki et al. «SymPy: Symbolic Computing in Python». En: *PeerJ Computer Science* 3 (2017), e103. DOI: 10.7717/peerj-cs.103 (vid. pág. 43).